

BETRIEBSSYSTEME

Technische Hochschule Mittelhessen

Andre Rein

– Betriebssysteme Prozessmanagement –

PROZESSE

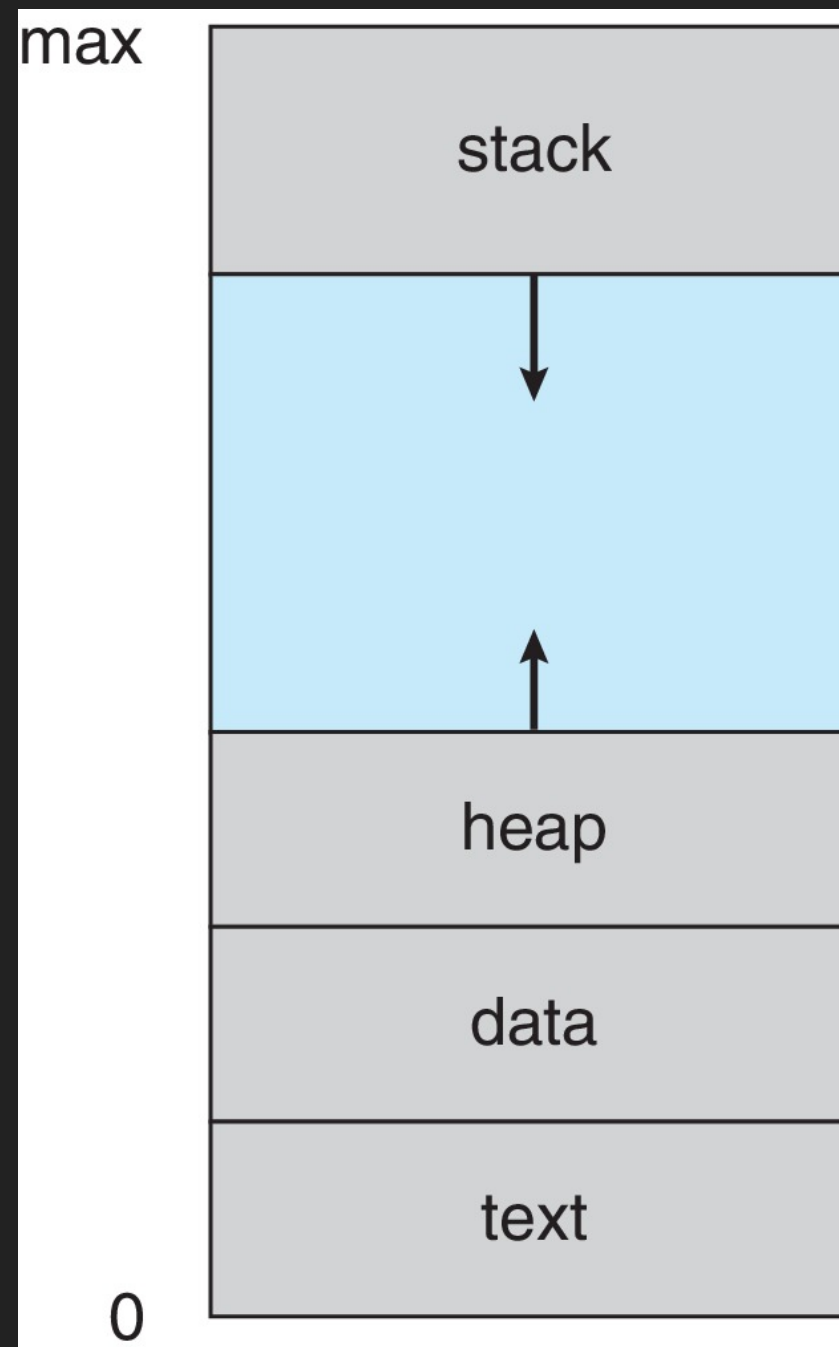


Frage:

Was ist der Unterschied zwischen einem Programm und einem Prozess?

- Ein Programm ist eine ausführbare Datei und befindet sich normalerweise auf einem sekundären Speicher (SSD/HDD)
- Ein Prozess ist ein Programm das in den Hauptspeicher geladen wurde und ausgeführt wird
- Aus Sicht des Betriebssystems ist ein Programm also etwas **passives** und ein Prozess etwas **aktives**

PROZESSE



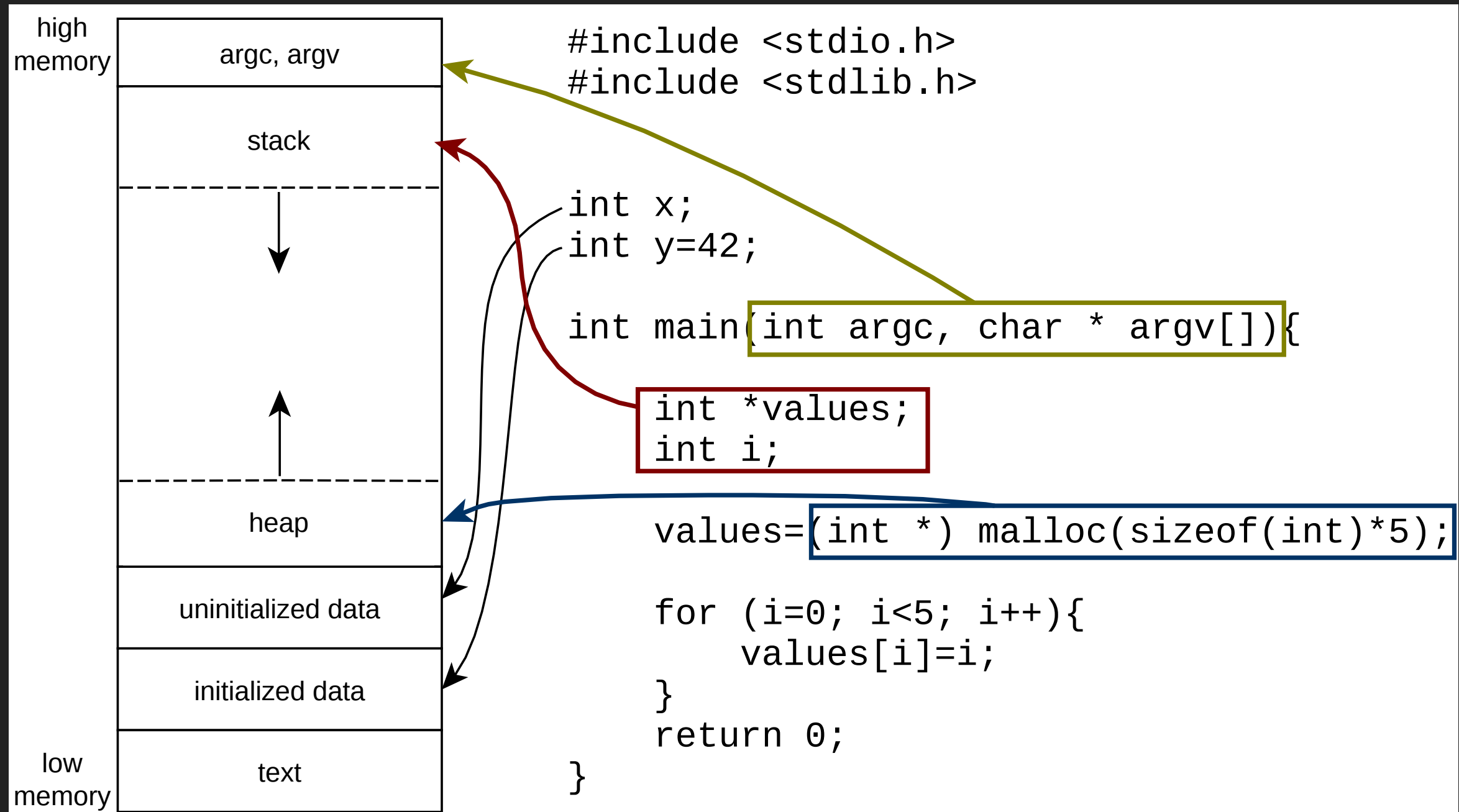
PROZESSE

- Vom Betriebssystem ausgeführte Programme werden als Prozesse bezeichnet
- Ein Prozess ist also ein Programm in Ausführung (das einer sequentiellen Ausführung folgt)
- Ein Prozess besteht aus vielen Teilen:
 - Programmcode → wird auch als *text section*, *text* oder *code* bezeichnet
 - Data → *data section*, *.data* → Speicher für globale Variablen
 - Stack mit temporären Daten von Funktionsaufrufen
 - Funktionsparameter, Rückgabewerten und lokalen Variablen (und andere Datenstrukturen)
 - Heap - Dynamischer und zur Laufzeit allozierter Speicherbereich für Variablen (und andere Datenstrukturen)
- Programmzähler (Program Counter [PC])

PROZESSE: PIDS

- Jeder Prozess hat eine Identifikationsnummer — Prozess ID (**PID**)
- Die PID ist das primäre Identifikationsmerkmal eines Prozesses und wird vom Kernel vergeben und verwaltet
 - Die PID ist eindeutig
 - Wird sequentiell für jeden neuen Prozess inkrementiert (früher: 32768; heute konfigurierbar unter Linux `sysctl kernel.pid_max pid_max` Maximum: $\leq 2^{22} \rightarrow 4194304$)
 - Die PID ist gültig bis zum Ende des Prozesses
 - Das kann u.U sehr kurz sein (weniges Mikrosekunden) oder aber sehr lange (bis zum Neustart)
 - Wird eine PID wieder frei, kann sie vom System wiederverwendet werden

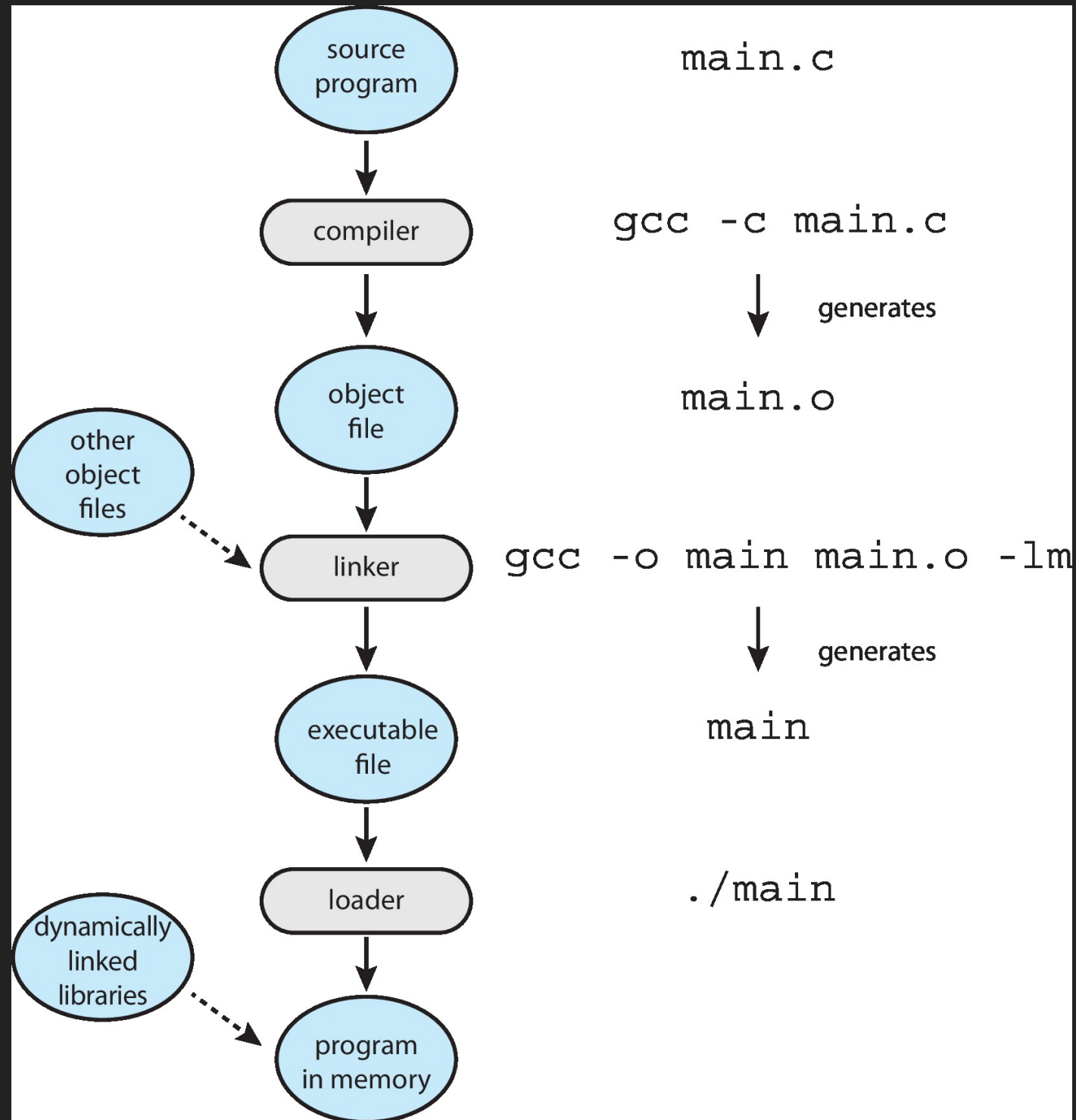
PROZESSE IM SPEICHER



LINKERS AND LOADERS [EINSCHUB]



LINKERS AND LOADERS



LINKERS AND LOADERS

- **Compiler:** Sourcecode wird in Speicher-ladbare Objektdateien (object files) kompiliert - reallokierbar (relocatable object files)
- **Linker:** Kombiniert verschiedene Objektdateien in eine ausführbare Programmdatei
 - Inklusive Bibliotheken bei statischem Prozess
- Das ausführbare Programm befindet sich nun auf einem sekundären Speicher (z.B. HDD,SSD)
- **Loader:** Laden des ausführbaren Programms (inklusive alle Abhängigkeiten) in den Speicher
 - Reallokierungsprozess weist hier die finalen Speicheradressen (vormals Platzhalter) im Programmcode zu
- Ausführung des Programms

LINKERS AND LOADERS

- Auf modernen Systemen werden normalerweise keine Bibliotheken während des Link-Prozesses in das Programm eingebunden
 - Dynamic Linked Libraries (DLLs - Windows `.dll`)
 - Shared Libraries (Linux/Unix `.so`)
- Während des Ladeprozesses werden die Bibliotheken vom Loader eingebunden/verfügbar gemacht
 - Alle Abhängigkeiten und Versionen sind im ausführbaren Programm angegeben
- Alle Programme haben Betriebssystem-spezifische Standardformate → somit weiß das Betriebssystem wie die Programme zu laden sind und wie sie gestartet werden

LINKERS AND LOADERS



Frage:

Welche Vorteile hat es für ein System geteilte Bibliotheken zu verwenden?

- Die Programme werden kleiner und verbrauchen somit weniger Platz auf der Festplatte
- Die Bibliotheken belegen nur einmalig Platz im Hauptspeicher
 - Gerade bei den Standardbibliotheken ist die Einsparung an Hauptspeicherplatz signifikant
- Bei einem Fehler/Bug in einer Bibliothek reicht es die Bibliothek zu aktualisieren
 - Bei statisch eingelinktem Code muss das Programm selbst aktualisiert werden (d.h. Neukompilierung und Verteilung)

LINKERS AND LOADERS

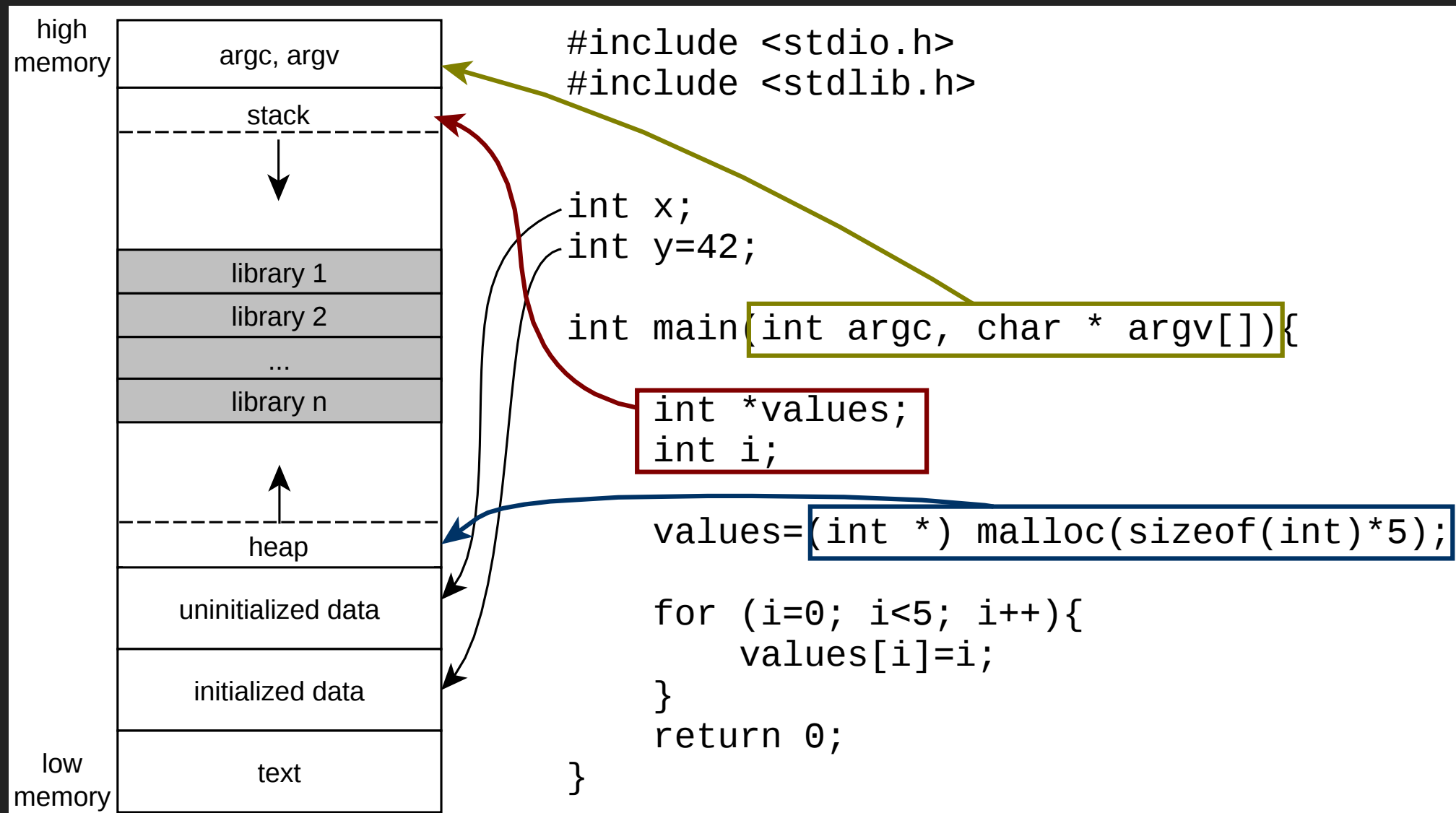


Frage:

Was denken Sie, gibt es auch Nachteile von geteilten Bibliotheken?

- Funktionsaufrufe in Programmen werden signifikant komplexer
 - Funktioniert hierbei über Sprungtabellen
 - Procedure Linkage Table (PLT) und Global Offset Table (GOT)
- Der Ladevorgang wird komplexer
 - Muss Abhängigkeiten beim Programmstart auflösen
 - (Und bei heutigen Systemen auch zur Laufzeit)

PROZESSE IM SPEICHER MIT SHARED LIBRARIES

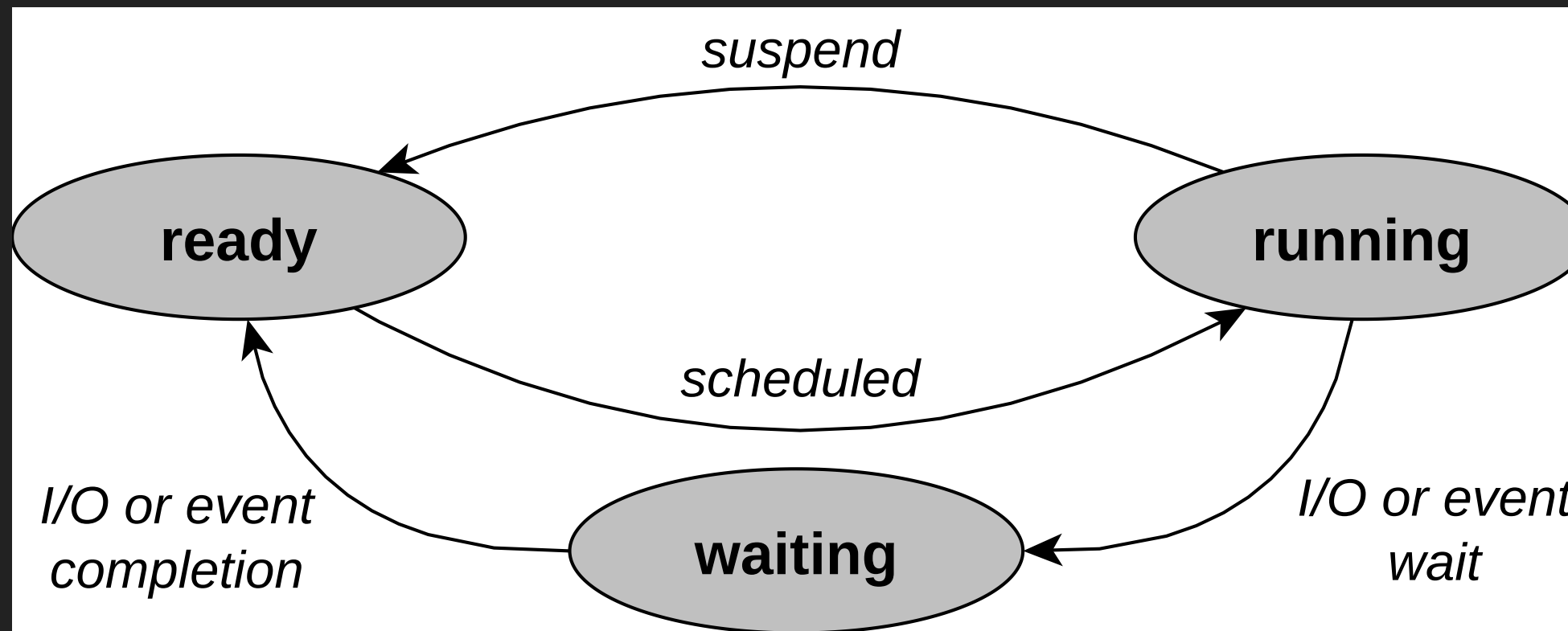


PROZESSE IM SPEICHER MIT SHARED LIBRARIES

	low	Address	Perm	Offset	Mapping	
	memory	-----				
		00400000	r-xp	00000000	main	<- points to the main file including text segment
		00600000	r--p	00000000	main	<- read only mapping (same as before)
		00601000	rw-p	00001000	main	<- data [initialized and uninitialized]
		00602000	rw-p	00000000	[heap]	<- our heap
						<-----
						note: much space in between
						<-----
		7fffffff030000	r-xp	00000000	libc-2.23.so	<-----library 1 start-----
		7fffffff1f0000	---p	001c0000	libc-2.23.so	
		7fffffff1f9000	---p	000001c9	libc-2.23.so	here libc (standard functions and syscalls)
		7fffffff3f0000	r--p	001c0000	libc-2.23.so	
		7fffffff3f4000	rw-p	001c4000	libc-2.23.so	<-----library 1 end-----
		7fffffff3f6000	rw-p	00000000		
		7fffffff400000	r-xp	00000000	ld-2.23.so	<-----library 2 start-----
		7fffffff403000	rwxp	00003000	ld-2.23.so	
		7fffffff407000	r-xp	00007000	ld-2.23.so	libld implements the loader for executables
		7fffffff415000	rwxp	00015000	ld-2.23.so	with shared libraries
		7fffffff625000	r--p	00025000	ld-2.23.so	
		7fffffff626000	rw-p	00026000	ld-2.23.so	<-----library 2 end-----
						<-----
						note: much space in between
						<-----
		7fffffff7ef000	rw-p	00000000	[stack]	<- our stack
high		7fffffffef000	r-xp	00000000	[vdso]	<- virtual dynamic shared object (implements some
memory		-----				syscalls in userspace e.g. time functions)

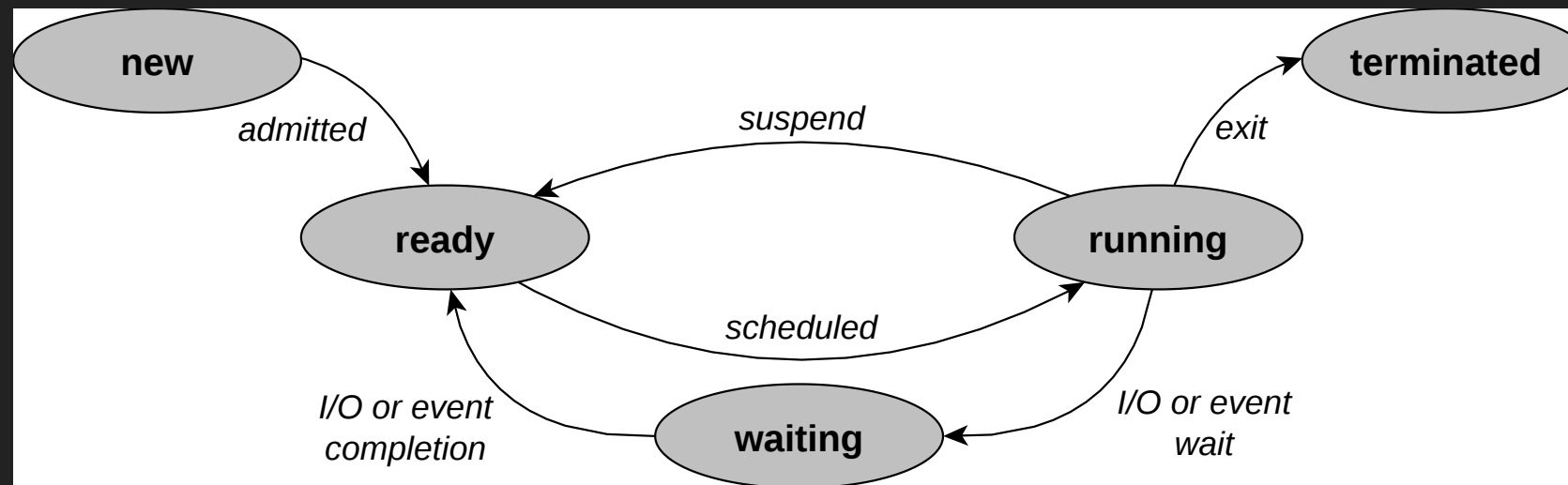
PROZESSZUSTÄNDE UND SCHEDULING

PROZESSZUSTÄNDE UND SCHEDULING



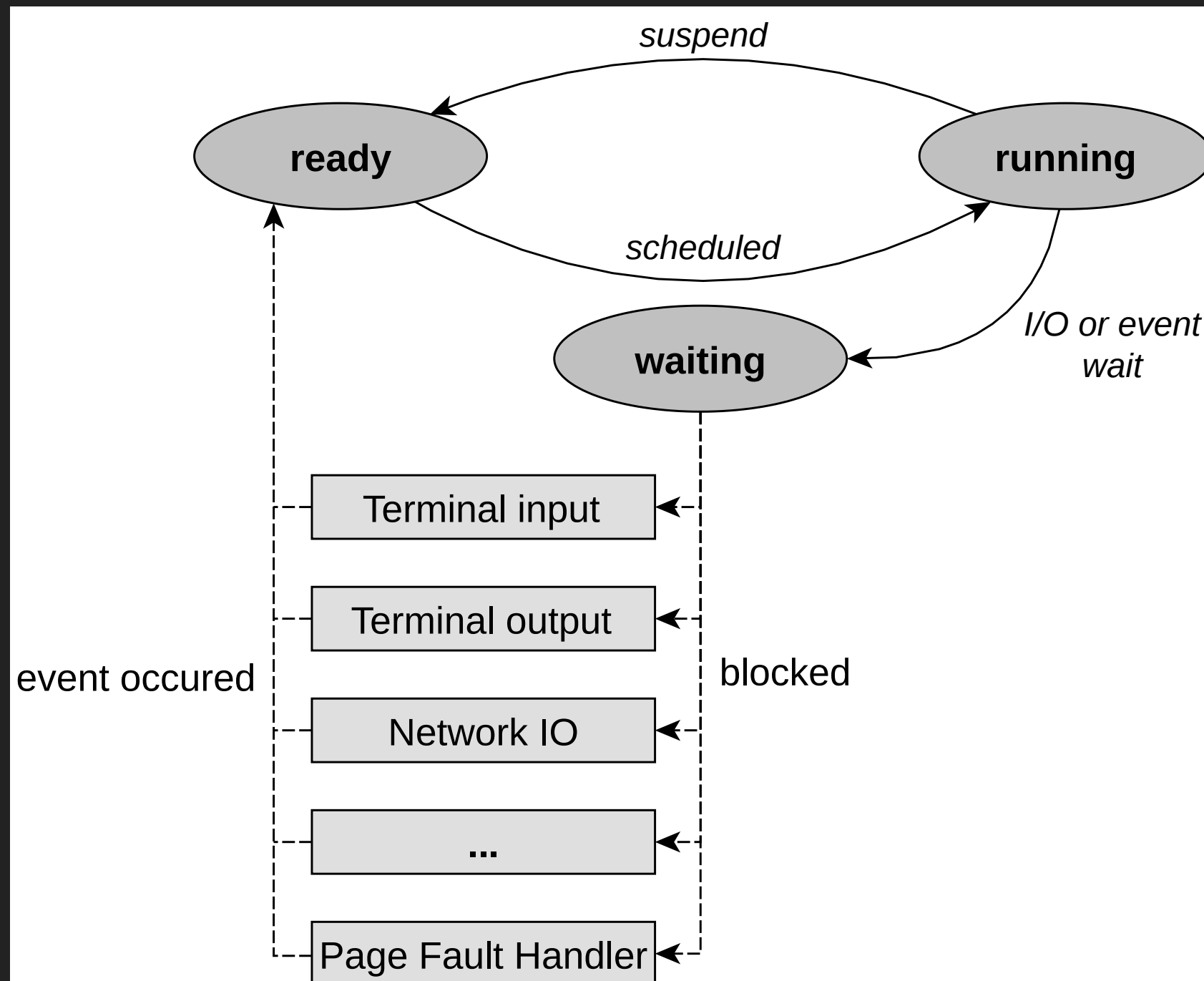
ready	bereit	Prozess wartet auf CPU Zuteilung
running	ausführend	Instruktionen werden von einer CPU ausgeführt
waiting	warten/blockiert	Prozess wartet auf ein Ereignis (z.B. Signal oder I/O Ende)

PROZESSZUSTÄNDE

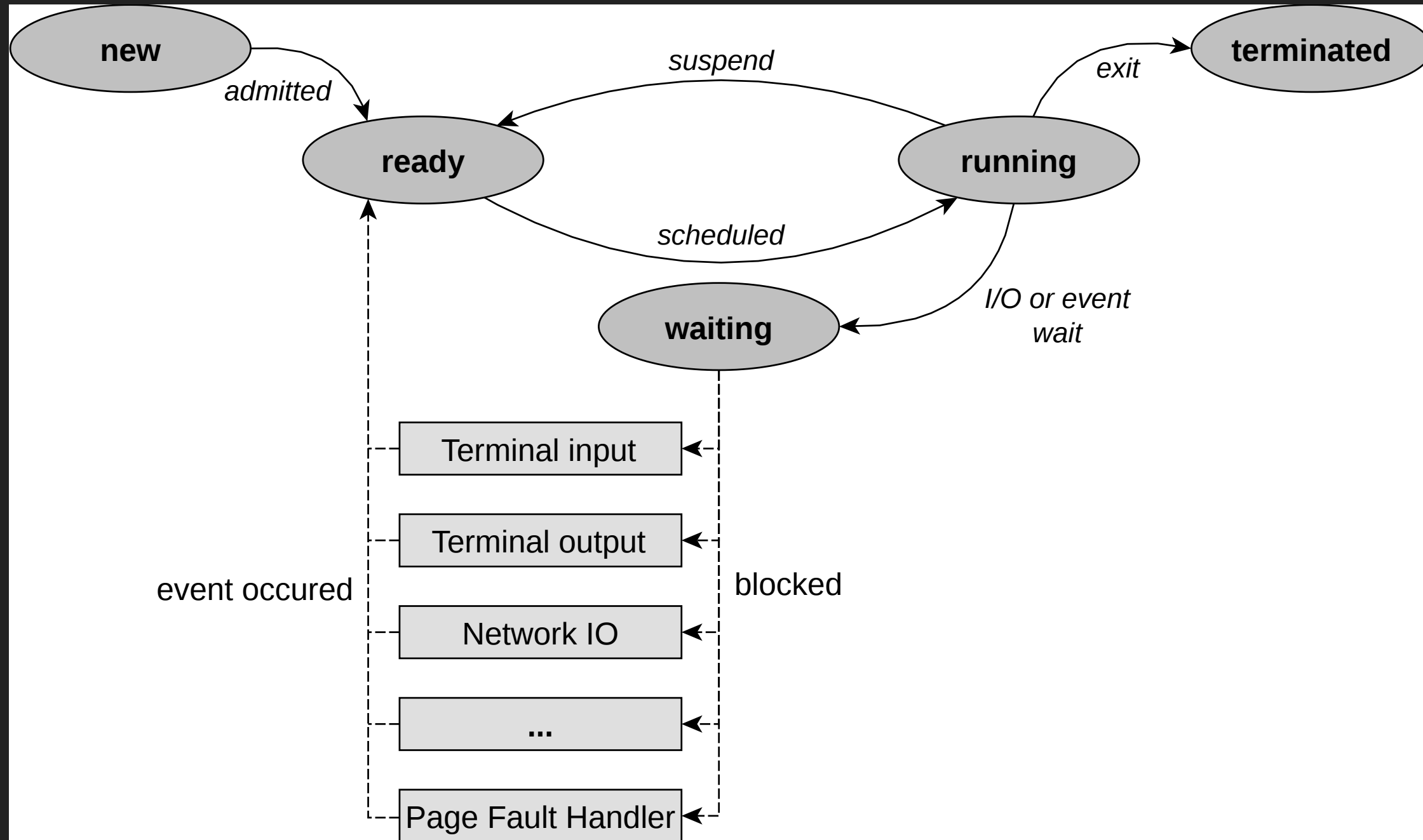


ready	bereit	Prozess wartet auf CPU Zuteilung
running	ausführend	Instruktionen werden von einer CPU ausgeführt
waiting	warten/blockiert	Prozess wartet auf ein Ereignis (z.B. Signal oder I/O Ende)
new	neu	Der Prozess wurde gerade erzeugt
terminated	beendet	Prozess hat Ausführung vollständig abgeschlossen

PROZESSZUSTÄNDE (MIT WARTEN)

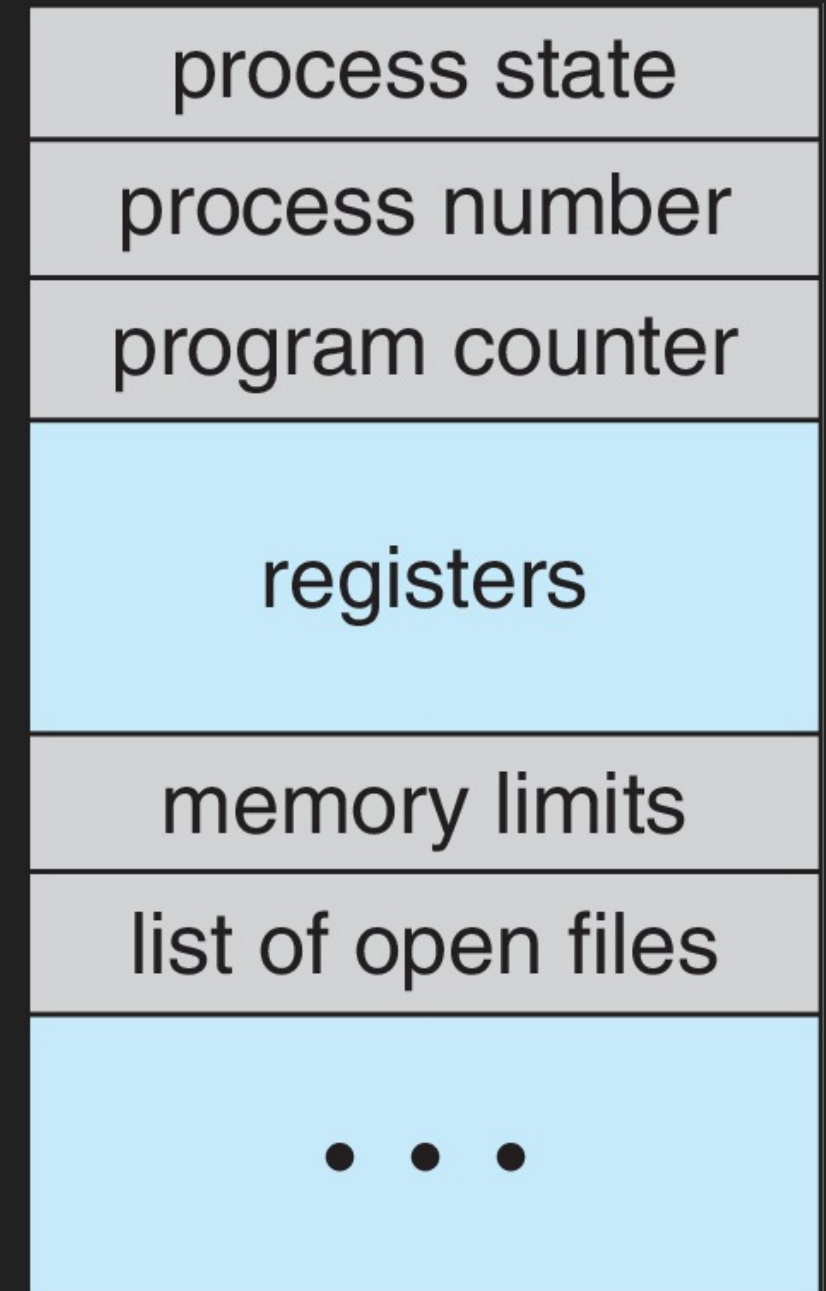


PROZESSZUSTÄNDE: GESAMTÜBERSICHT



PROZESSKONTROLLBLOCK (PCB)

- **Prozesszustand** - running, waiting, ready
- **Prozessnummer (PID) & Programmzähler** - Adresse der als nächstes auszuführenden Instruktion
- **Register** - Speicherplatz für Inhalte aller relevanten Register (Hardwarearchitekturabhängig)
- **CPU-Scheduling Information** - Priorität, Zeiger auf Queues
- **Speichermanagement Information** - Allokierter Speicher für den Prozess
- **Accounting** - Verbrauchte CPU Zeit, Laufzeit
- **I/O Information** - I/O Geräte, Liste aller geöffneten Dateien

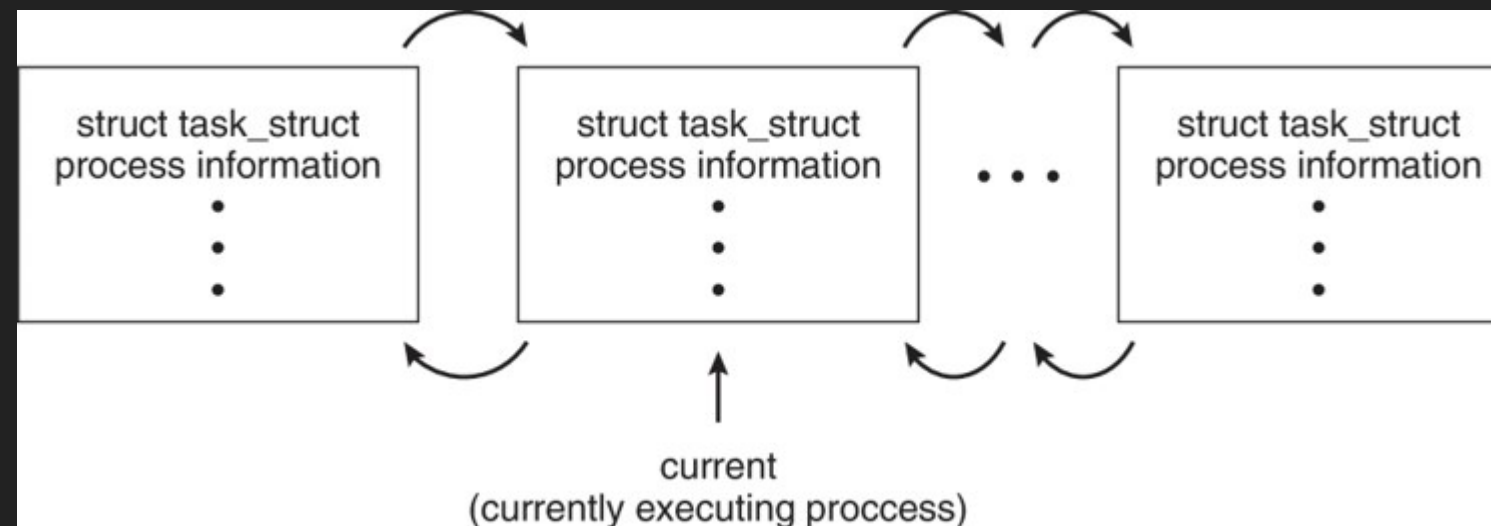


PROZESSKONTROLLBLOCK (LINUX)

Ausschnitt `task_struct` aus `<include/linux/sched.h>`

```
struct task_struct {  
    pid_t pid;           /* process identifier */  
    long state;          /* state of the process */  
    unsigned int time_slice /* scheduling information */  
    struct task_struct *parent; /* this process's parent */  
    struct list_head children; /* this process's children */  
    struct files_struct *files; /* list of open files */  
    struct mm_struct *mm; /* address space of this process */  
    ...  
}
```

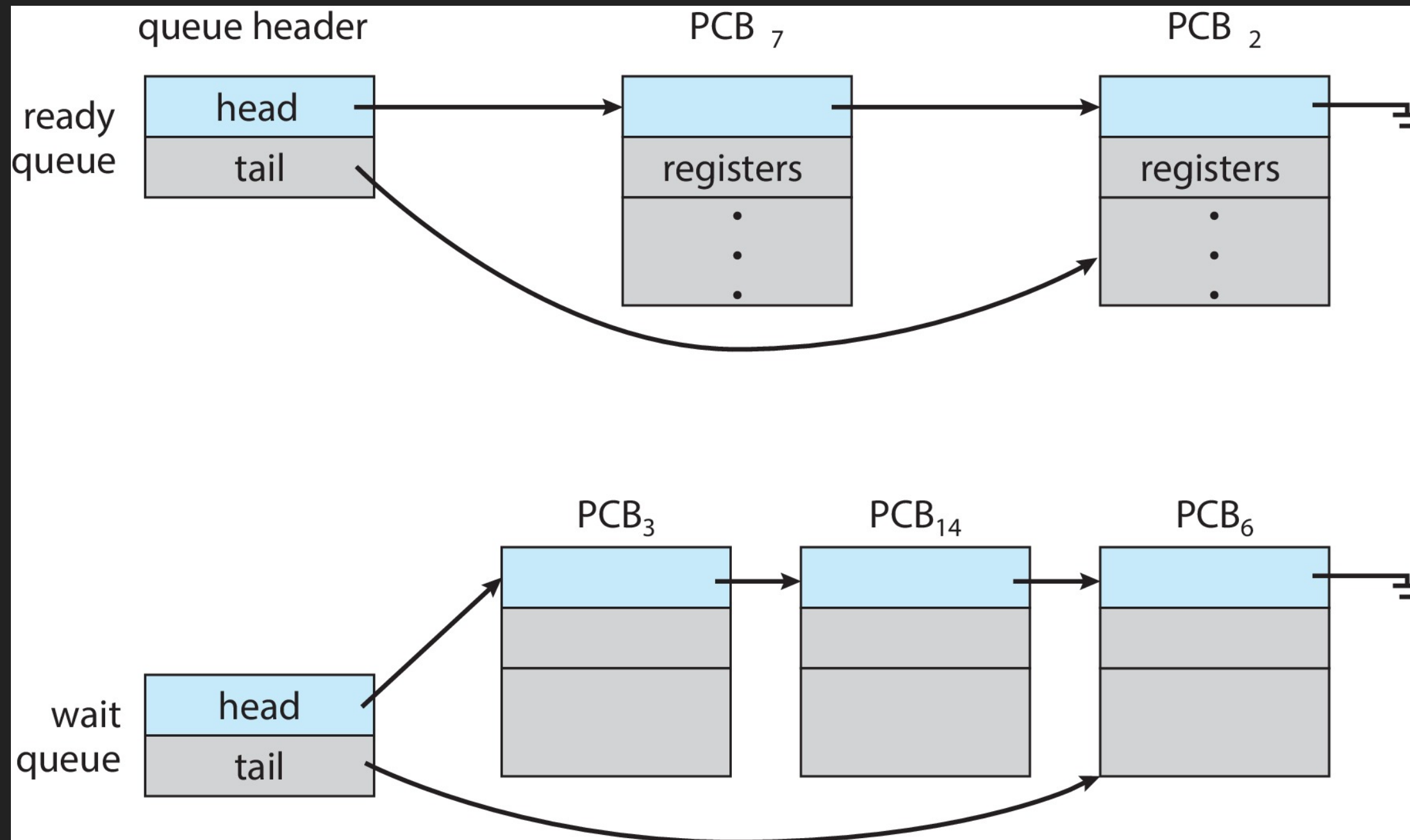
Doppelt verkettete Liste!



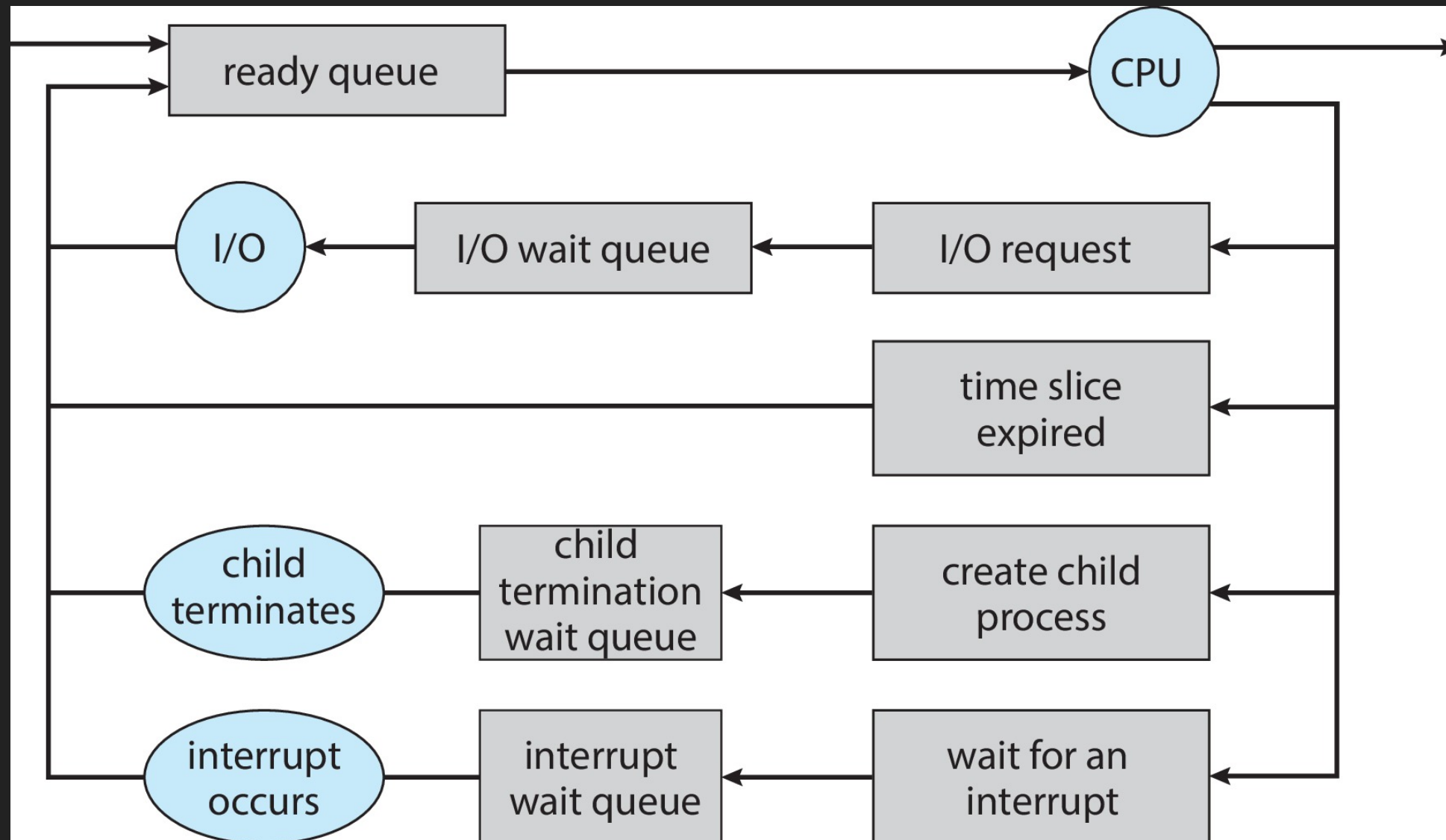
PROZESS SCHEDULING

- Sinn: Ausnutzung der CPU maximieren - Schnelle Zuteilung eines Prozesses auf einen CPU-core
- Der Prozess Scheduler wählt den nächsten Prozess aus der ausgeführt werden soll
 - Verwaltet sog. **scheduling queues** von Prozessen
 - **ready queue** - Liste aller Prozesse, die sich im Hauptspeicher befinden und zur Ausführung bereit sind und warten.
 - **wait queues** - Listen von Prozessen die auf Ereignisse warten (z.B. I/O)
- Alle Prozesse wechseln ständig zwischen diesen beiden Queues

WAIT UND READY QUEUES



PROZESS SCHEDULING MIT WARTEN



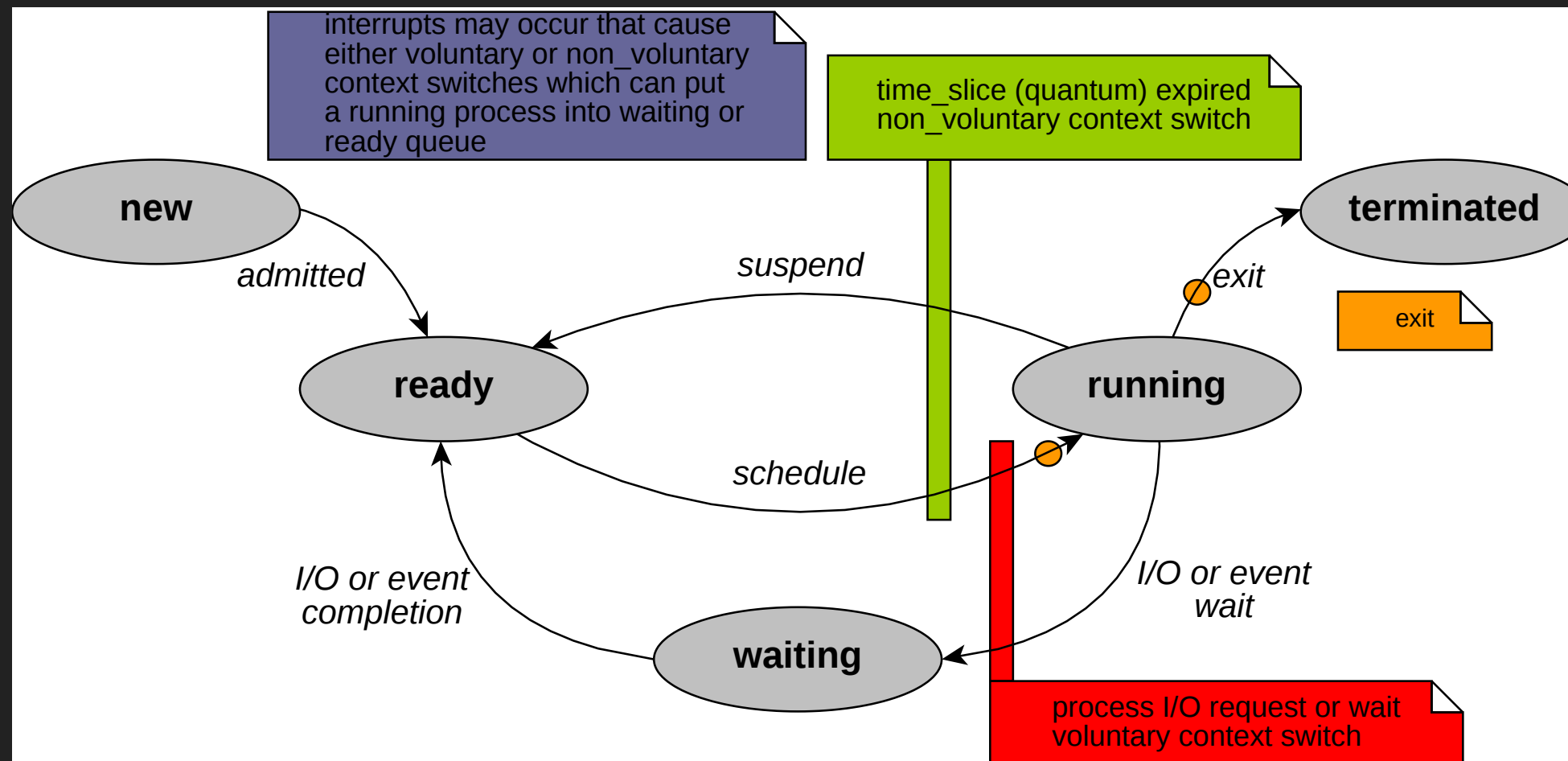
KONTEXTWECHSEL

Wenn die CPU zu einem anderen Prozess wechselt, muss das System den Zustand des alten Prozesses speichern und den gespeicherten Zustand für den neuen Prozess über einen sog. Kontextwechsel laden

- Der **Kontext** eines Prozesses ist Teil des **PCB**
- Je komplexer ein Betriebssystem und der PCB ist, desto länger dauert ein Kontextwechsel
 - Kontextwechselzeit ist reiner Overhead
 - Messungen bei Linux in 2018 — 1-2 μs (Mikrosekunden) (**context switch time**)
- Zeitabhängig auch vom Hardware-Support:
 - Mehrere Register-Sets pro CPU - Register müssen seltener in Hauptspeicher gespeichert werden
 - Instruktion die alle Register auf einmal speichert

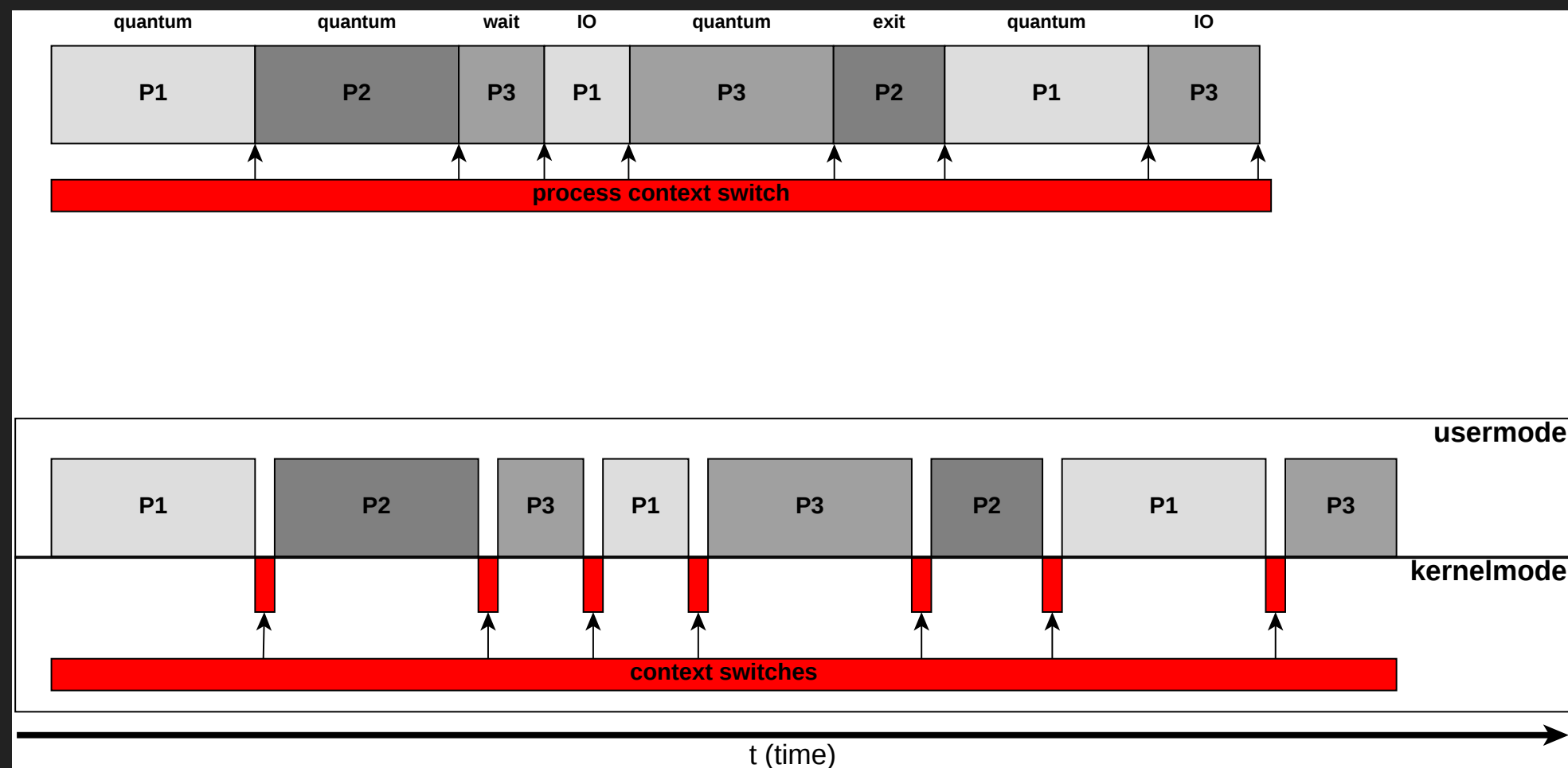
KONTEXTWECHSEL: PROZESSZUSTÄNDE

- Kontextwechsel können im Prozessmodell zu unterschiedlichen Zeitpunkten erfolgen
 - $\text{ready} \leftarrow \rightarrow \text{running (quantum)}$, $\text{running} \leftarrow \rightarrow \text{waiting (I/O)}$, $\text{running} \leftarrow \rightarrow \text{terminated (exit)}$

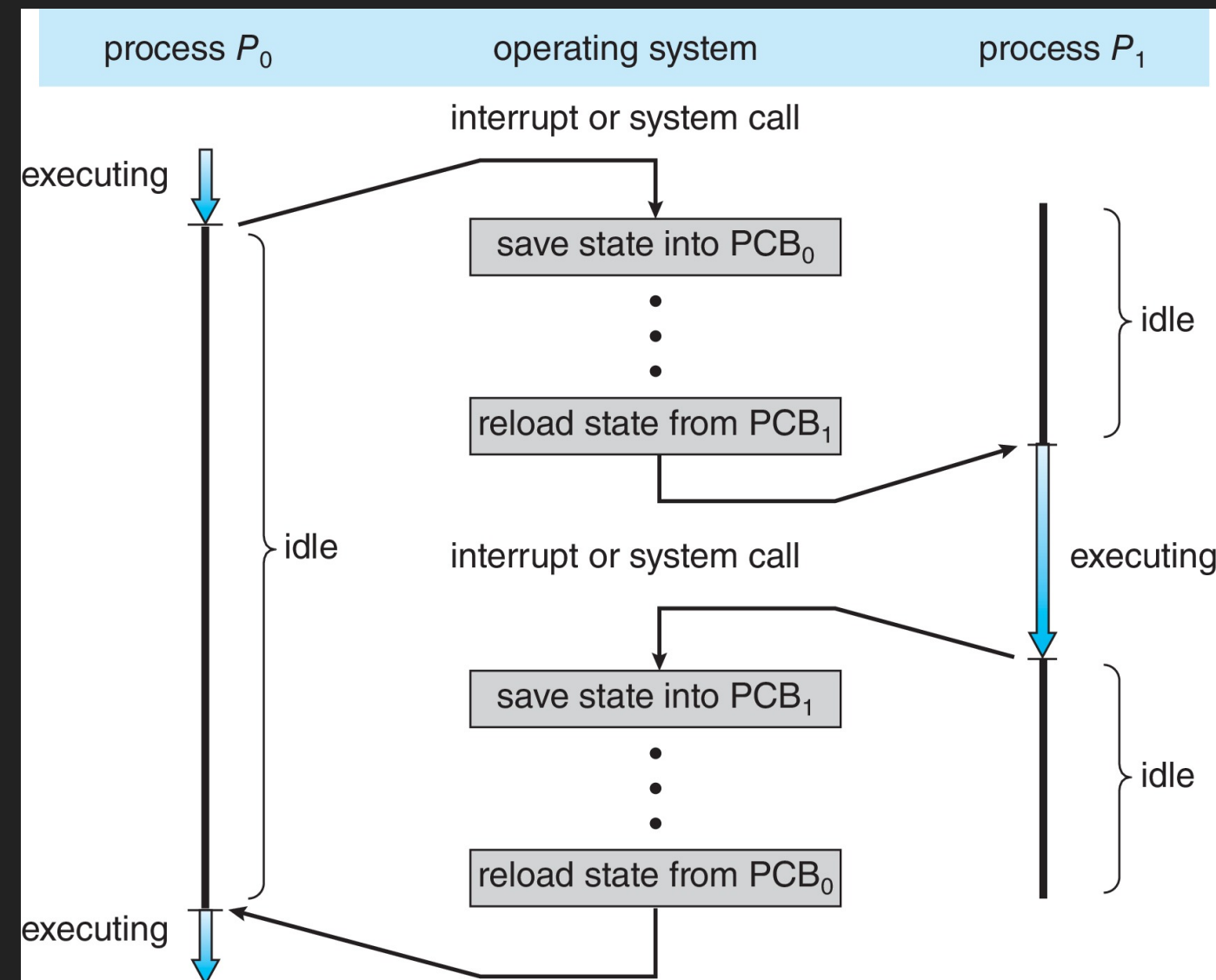


KONTEXTWECHSEL: KONTEXTWECHSEL KERNEL

- Ein Kontextwechsel erfolgt wenn das BS von einem zu einem anderen Prozess wechselt → Quantum abgelaufen, I/O-Operation, Prozess wartet oder Interrupt
- Der Kontextwechsel selbst benötigt eine feste Zeitspanne an Overhead
 - Der eigentliche Kontextwechsel erfolgt im Betriebssystem im Kernelmodus



KONTEXTWECHSEL: PROZESSKONTROLLBLOCK



KONTEXTWECHSEL: INFORMATION LINUX

- In Linux kann man sich zu jedem Prozess anzeigen lassen wie oft ein Kontextwechsel für den jeweiligen Prozess ausgeführt wurde
- Die Information dazu ist in `/proc/<pid>/status`, welche mit `cat` ausgegeben werden kann
 - Die Informationen dazu stammen aus dem PCB `task_struct`

BEISPIEL

```
$cat /proc/53547/status | grep ctxt
voluntary_ctxt_switches:      5047
nonvoluntary_ctxt_switches:   165
$#kurze zeit warten
$cat /proc/53547/status |grep ctxt
voluntary_ctxt_switches:      5065
nonvoluntary_ctxt_switches:   165
```

KONTEXTWECHSEL: EXPERIMENT

Wir schauen uns die Kontextwechsel für einen aktiven Prozess an. Beispiel hier `gnome-shell` der unsere GUI in der VM bereitstellt.

```
$watch -c -n 1 'cat /proc/$(pidof gnome-shell)/status |egrep "Stat|ctx"'

Every 1,0s: cat /proc/$(pidof gnome-shell)/status |egrep "Stat|ctx"      lunar: Mon Oct 28 21:04:18 2019

State:  S (sleeping)
voluntary_ctxt_switches:      2837541
nonvoluntary_ctxt_switches:  155757
```

Bewegen Sie die Maus und achten Sie darauf welcher Zähler sich erhöht

- Normalerweise bleibt der Zähler `nonvoluntary_ctxt_switches` relativ konstant. Sobald man aber die Maus bewegt (Interrupts + Mauszeiger auf Bildschirm bewegen) sieht man wie sich der `nonvoluntary_ctxt_switches` auf einmal erhöht.

KONTEXTWECHSEL: SPEICHERUNG REGISTER LINUX X86_64

```
// %rdi: prev task
// %rsi: next task
ENTRY(__switch_to_asm)
    /* Save callee-saved registers */
    pushq %rbp
    pushq %rbx
    pushq %r12
    pushq %r13
    pushq %r14
    pushq %r15

    /* switch stack */ /*TASK_threadsp is byte offset to SP field in task_struct*/
    movq %rsp, TASK_threadsp(%rdi) /* -> %rdi+12120 */
    movq TASK_threadsp(%rsi), %rsp /* -> %rsi+12120 */

    /* restore callee-saved registers */
    popq %r15
    popq %r14
    popq %r13
    popq %r12
    popq %rbx
    popq %rbp

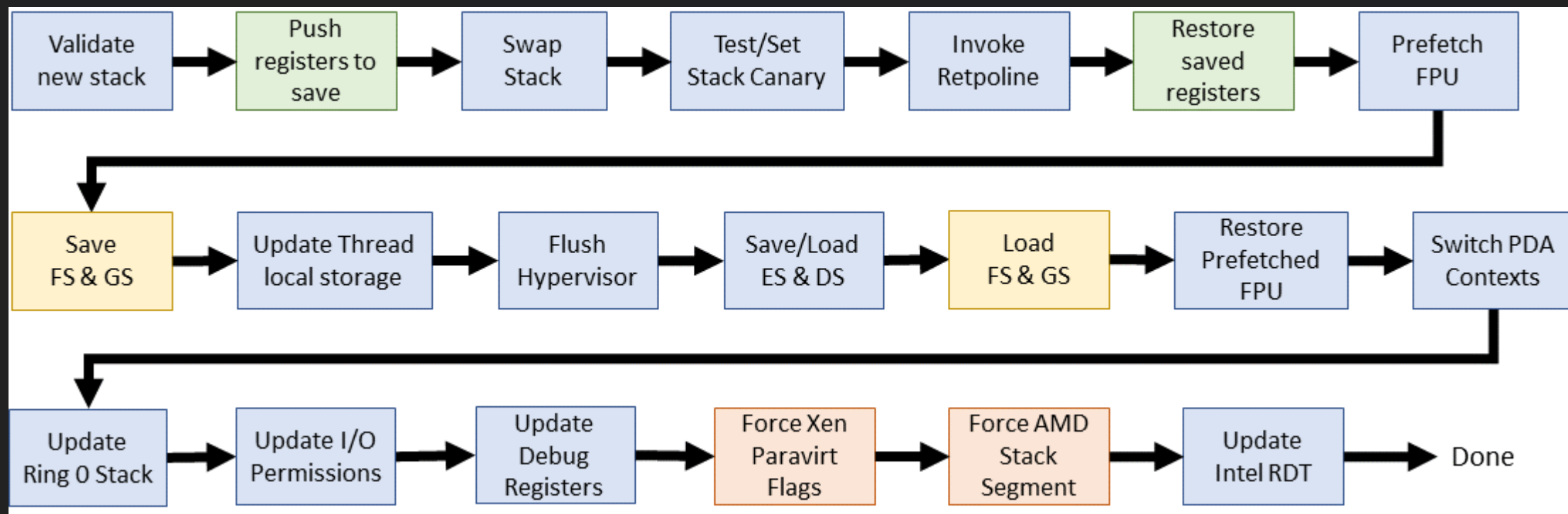
    jmp __switch_to
END(__switch_to_asm)
```

Leicht bereinigte Version von https://github.com/torvalds/linux/blob/master/arch/x86/entry/entry_64.S

AT&T Assembler Syntax: <https://csiflabs.cs.ucdavis.edu/~ssdavis/50/att-syntax.htm>

KONTEXTWECHSEL: LINUX FURTHER READING

- Nach dem Kopieren der Register müssen noch weitere Aufgaben erledigt werden
 - Details → [evolution_x86_context_switch_linux](#)

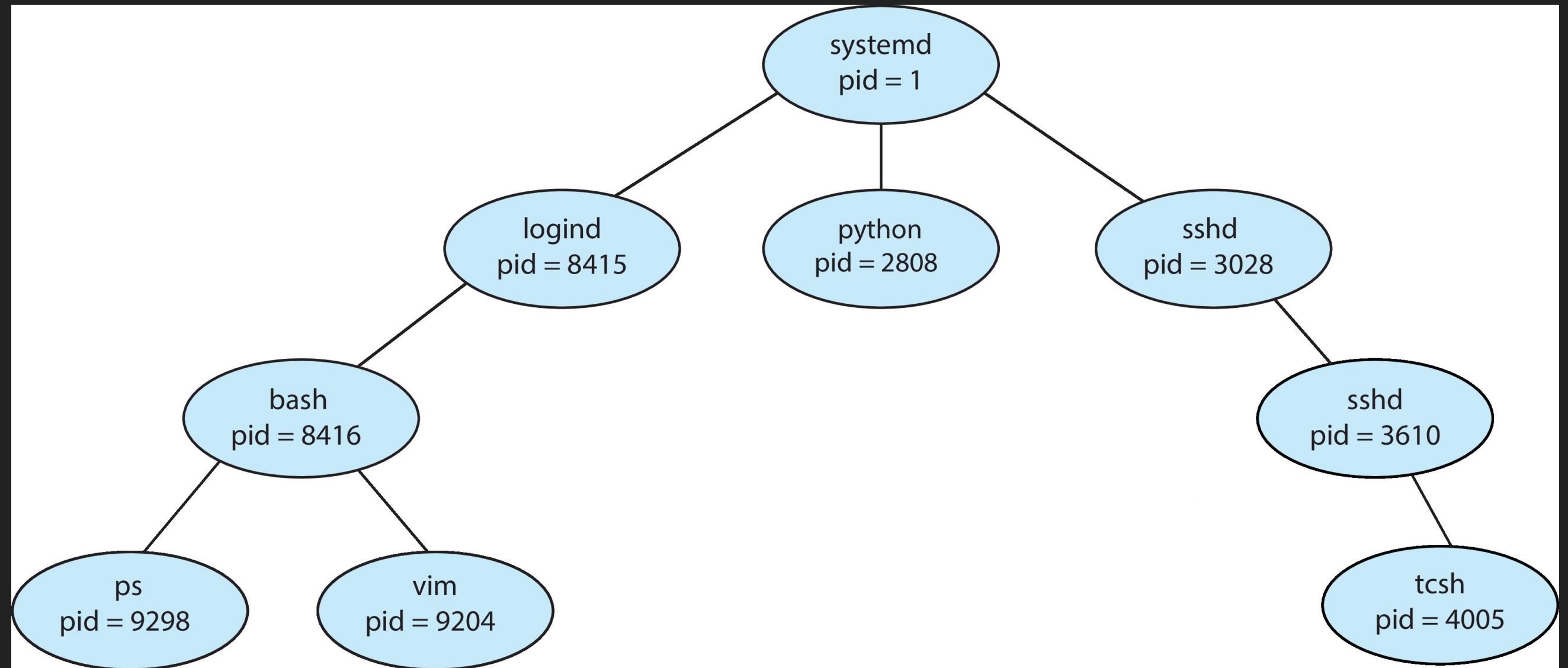


- Diese Aufgaben werden in Linux in der Funktion `__switch_to` erledigt
 - `__switch_to` Implementierung

PROZESSERZEUGUNG: GRUNDKONZEPT

- Elternprozess erzeugt Kindprozesse, die wiederum andere Prozesse erzeugen
 - Es entsteht ein Baum von Prozessen
- Ein Prozess wird über seine PID identifiziert und verwaltet
- Optionen für die gemeinsame Nutzung von Ressourcen
 - Eltern und Kinder teilen sich alle Ressourcen
 - Kinder teilen sich eine Teilmenge der Ressourcen der Eltern
 - Eltern und Kinder teilen sich keine Ressourcen
- Ausführungsoptionen
 - Eltern und Kinder werden gleichzeitig ausgeführt
 - Eltern warten, bis Kinder terminieren

PROZESSERZEUGUNG: PROZESSBAUM LINUX



PROZESSERZEUGUNG MIT FORK UND EXEC

- Passen Sie bitte jetzt sehr gut auf!
- Sie werden die folgenden Beispiele in der ersten Hausübung anwenden und erweitern!
- Ihnen muss klar sein wie das Zusammenspiel der Funktionen/System Calls funktioniert.

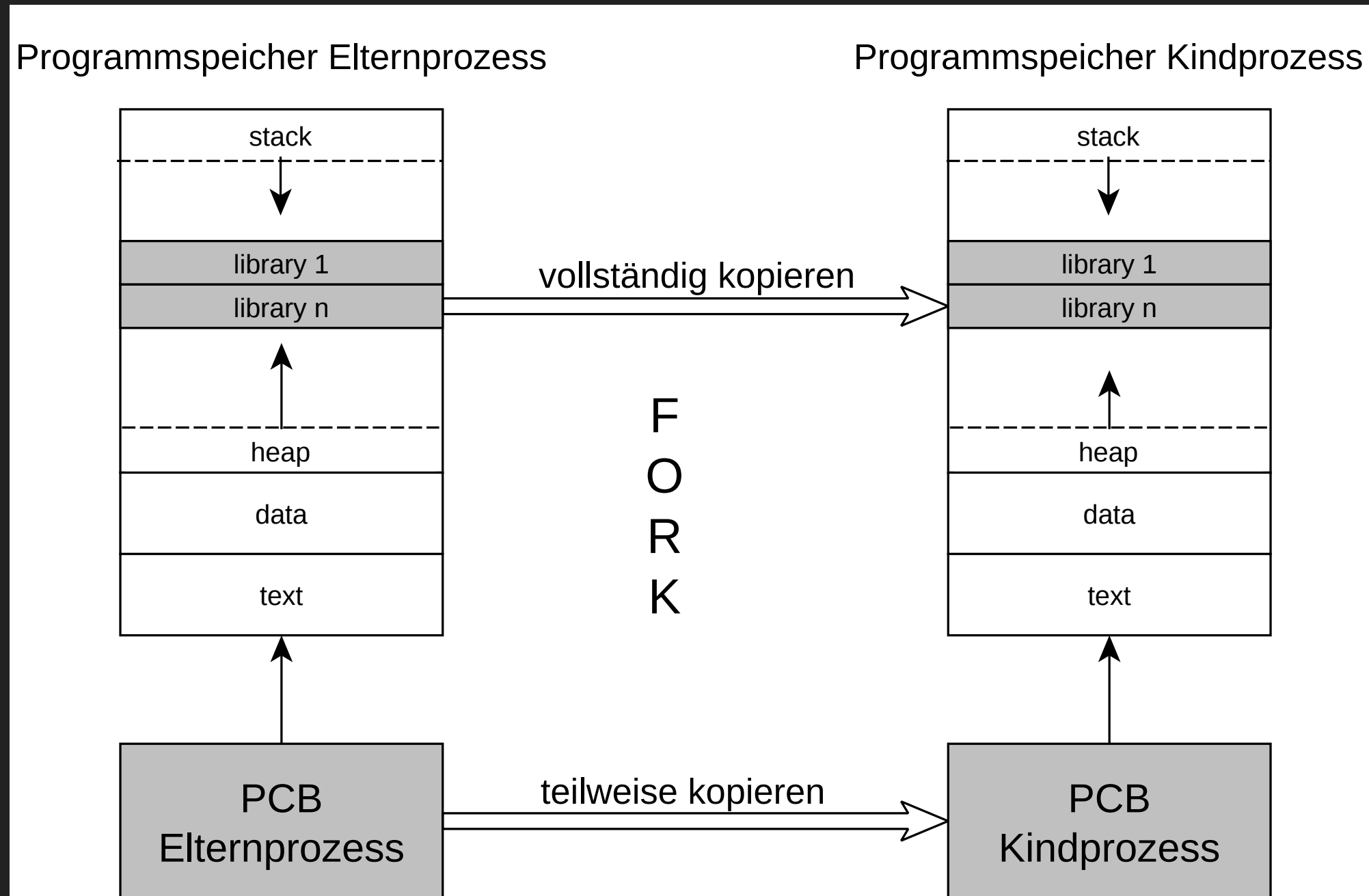
Bei Unklarheiten bitte nachfragen!

!!!TAFEL!!!

PROZESSERZEUGUNG MIT FORK UND EXEC

- Der system call `fork()` erzeugt einen neuen Kindprozess
 - Der Speicher des neu erzeugten Kindprozesses ist ein (nahezu) exaktes Abbild des Elternprozesses
- Kindprozess: Der system call `exec()` wird nach `fork()` aufgerufen
 - Der Speicher wird ersetzt mit dem in `exec()` übergebenen Programms
- Elternprozess: Der system call `wait()` wird nach `fork()` aufgerufen
 - Wartet darauf das der Kindprozess via `exit()` terminiert

FORK: KOPIE VON SPEICHER UND PCB



FORK: KOPIE VON SPEICHER UND PCB

- Vom Elternprozess vererbte Attribute (Auswahl):
 - Deskriptoren von offenen Dateien und Kommunikationsendpunkten (sockets, pipes)
 - Benutzerinformationen
 - Prozessorzustand (Register) → leicht anders (`return value`)
 - Priorität (nice-Wert)
- Vom Elternprozess nicht vererbte Attribute (Auswahl):
 - PID (wird neu vergeben)
 - PPID = PID von Elternprozess
 - Erzeugungszeit
 - Accountinginformation (Zeit CPU-Nutzung)

FORK

fork.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    pid_t  pid;
    if ((pid = fork()) == -1) {
        /* error */
        perror("Fork failed: ");
        exit(1);
    } else if (pid == 0) {
        /* child */
        printf("child:  PID [%i] PPID [%i]\n", getpid(), getppid());
        exit(0);
    } else {
        /* parent */
        printf("parent: PID [%i] CPID [%i]\n", getpid(), pid);
    }
}
```

```
$gcc -o fork fork.c && ./fork
parent: PID [6040] CPID [6041]
child:  PID [6041] PPID [6040]
$./fork
parent: PID [8839] CPID [8840]
child:  PID [8840] PPID [8839]
```


FORK ???

fork1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    pid_t pid;
    int i=10;
    if ((pid = fork()) == -1) {
        /* error */
        perror("Fork failed: ");
        exit(1);
    } else if (pid == 0) {
        /* child */
        printf("child:  PID [%i] PPID [%i]\n", getpid(), getppid());
        for (;i<50;i++){
            printf("[child] executing i=%i\n", i);
        }
        exit(0);
    } else {
        /* parent */
        printf("parent: PID [%i] CPID [%i]\n", getpid(), pid);
        for (;i<50;i++){
            printf("[parent] executing i=%i\n", i);
        }
    }
}
```

- Welche Ausgabe erwarten Sie?
 - Was kommt zuerst?
 - Was kommt zuletzt?
- Wie verhält es sich mit `i`?
 - Wie viele Ausgaben wird das Programm in der Schleife machen?
 - 40? 20? X?

FORK ERROR

```
if ((pid = fork()) == -1 ) {  
    /* error */  
    perror("Fork failed: ");  
    exit(1);  
}
```

- Der `fork()`-Aufruf kann auch Fehlschlagen
 - Prozesslimit für Benutzer erreicht
 - Ungenügender Speicher für Programmspeicher oder Deskriptor
- Fehlerbehandlung im Programm:
 - `fork()` Rückgabewert < 0
 - Setzt automatisch globale `errno` (`errno.h`) Variable
 - Funktion `perror()` gibt eine Fehlermeldung zum Fehlercode aus — z.B. `Fork failed: cannot allocate memory`

Mit `exit(1)`; terminiert der aktuelle Prozess und signalisiert dem Elternprozess einen fatalen Fehler

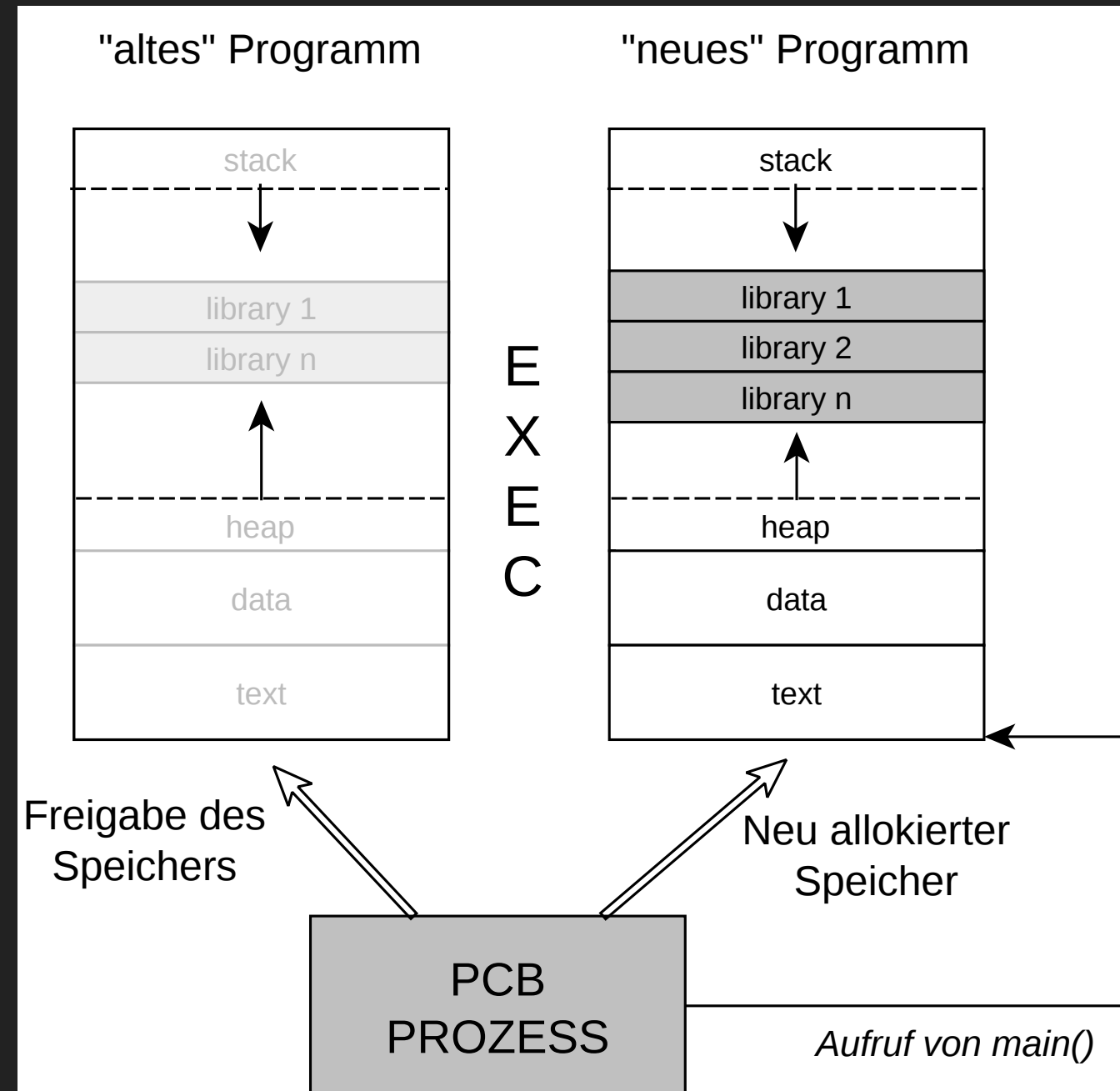
exec SYSTEM CALL

- `exec()` ersetzt das aktuell ausgeführte Programm mit einem anderen Programm
 - Ein Prozess kann also zur Laufzeit nacheinander andere Programm laden und ausführen
- Der Einsprungpunkt im neuen Programm ist `main()`
- Es erfolgt kein Rücksprung in das ursprüngliche Programm!
- Dateideskriptoren bleiben erhalten

Parameter:

- Dateipfad des aufzurufenden Programms z.B. `/bin/lis`
- Liste der zu übergebenen Parameter z.B. `-l -a`
- *Liste der Umgebungsvariablen (optional)*

EXEC: FREIGABE, LADEN, AUFRUF



EXEC: VARIANTEN

```
1 #include <unistd.h>
2
3 int execl(const char *path, const char *arg, ... ①
4           /* (char *) NULL */);
5 int execlp(const char *file, const char *arg, ... ① ②
6           /* (char *) NULL */);
7 int execl_e(const char *path, const char *arg, ... ① ③
8             /*, (char *) NULL */ , char * const envp[]);
9 int execv(const char *path, char *const argv[]); ④
10 int execvp(const char *file, char *const argv[]); ④ ②
11 int execvpe(const char *file, char *const argv[], ④ ③
12             char *const envp[]);
```

Varianten-Auswahl mit Buchstaben hinter `exec` (l, v, p, e)

- ① l → Übergabe der Programmparameter als `NULL`-terminierte Liste
- ② p → Sucht implizit nach Programmname in Pfaden (falls kein `/` in Namen vorhanden ist)
- ③ e → Ermöglicht die Übergabe eines Vektors für Umgebungsvariablen
- ④ v → Übergabe der Programmparameter als Vektor



Bei Aufrufen ohne `e`-Variante werden standardmäßig alle Umgebungsvariablen des aufrufenden Prozesses übergeben

EXEC: `exec1` VS. `execv`

`exec1()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    exec1("/bin/ls", "ls", "-l", "-a", "/tmp", NULL);
    /* remember: exec calls only return on errors! */
    perror("Command failed: ");
    exit(1);
}
```

- Bei `exec1()` wird jedes an das Programm zu übergebende Argument einzeln übergeben, ein `NULL`-String terminiert die variabel lange Liste

`execv()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    char *arguments[]={"ls", "-l", "-a", "/tmp", NULL};
    execv("/bin/ls", arguments);
    /* remember: exec calls only return on errors! */
    perror("Command failed: ");
    exit(1);
}
```

- Bei `execv()` wird ein Vektor-Array übergeben, die `char*`-Elemente sind die an das Programm zu übergebenden Zeichenketten
 - Letztes Argument muss ebenfalls ein `NULL`-Element sein!

EXEC: `execvp` VS. `execvpe`

`execvp()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    char *arguments[]={"ls", "-l", "-a", "/tmp", NULL};
    execvp(arguments[0], arguments);
    /* remember: exec calls only return on errors! */
    perror("Command failed: ");
    exit(1);
}
```

- `execvp()` ist äquivalent zum Aufruf von `execv()` mit dem Unterschied das im ersten Parameter kein expliziter Pfad zum Programm angegeben werden muss
 - Standardmäßig werden die in der Umgebungsvariable `PATH` (`PATH=/bin:/sbin:/usr/bin:/usr/sbin`) angegebenen Verzeichnisse nach dem Programm durchsucht

`execvpe()`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main(void){
    char *arguments[]={"env", NULL};
    char *environment[]={"FOO=BAR", NULL};
    execvpe(arguments[0], arguments, environment);
    /* remember: exec calls only return on errors! */
    perror("Command failed: ");
    exit(1);
}
```

- `execvpe()` erlaubt zusätzlich Umgebungsvariablen als dritten Parameter zu übergeben
 - !Achtung!: Hierbei werden ausschließlich die explizit angegebenen Umgebungsvariablen gesetzt
 - !Achtung!: `#define _GNU_SOURCE` muss vor `#include<unistd.h>` angegeben werden

MÖGLICHE FEHLER BEI PROGRAMMAUFRUF VIA `exec()`

- E2BIG – Argumentliste zu groß
- EACCESS – Zugriffsrecht „ausführbar“ fehlt
- ENOMEM – Hauptspeichermangel
- ENOENT – Datei nicht gefunden (NO directory ENTry)

SONDERFALL: `system()`

Eine weitere Funktion zum Aufruf von Programmen ist `system()`, was hier zur Vollständigkeit angegeben ist! Bitte verwenden Sie `system()` nicht in den Übungen!

- Verwendet implizit `fork()` und `exec()`
- Übergabeparameter ist ein einfacher String:
 - `system("/bin/ls -l -a");`
- `system()` kehrt im Gegensatz zu `exec()` zum aufrufenden Programm zurück
- Ist in `<stdlib.h>` definiert

PROZESSTERMINIERUNG UND SIGNALE (1)

Ein Prozess terminiert sich selbst via `exit(exitcode)`:

- `exit(0)` oder `exit(EXIT_SUCCESS)` — Erfolgreicher Aufruf, alles OK
- `exit(1)` oder `exit(EXIT_FAILURE)` -- Fataler Fehler aufgetreten
- Kernel gibt belegte Ressourcen frei:
 - Programmspeicher
 - Deskriptoren Dateien und Kommunikationsendpunkte
- Erhalten bleibt:
 - Der `exitcode`-Wert
 - Ressourcenverbrauchswerte (verbrauchte CPU-Zeit, gesamte Laufzeit, etc.)

PROZESSTERMINIERUNG UND SIGNALE (2)

- Für `exit()` lässt sich via `int atexit(void (*function)(void));` eine Funktion als exit-handler registrieren
 - z.B. könnte man hier temporär angelegte Dateien löschen
 - Bei der Verwendung von `_exit()` terminiert der Prozess ohne den Aufruf eines exit-handlers
- Prozess erhält Signal
 - Abhängig vom Signal wird der jeweilige signal-handler aufgerufen
 - z.B. `Strg+c` (`SIGINT`) terminiert den Prozess
 - Für alle Signale können eigene signal-handler registriert werden
 - `signal(int signum, sighandler_t handler);`

WARTEN: `waitpid()` UND `wait()`

- Das warten auf Kindprozesse wird mit `wait()` bzw. `waitpid()` realisiert

```
1 #include <sys/types.h>
2 #include <sys/wait.h>
3
4 pid_t wait(int *status);
5 pid_t waitpid(pid_t pid, int *status, int options);
```

Der Aufruf von `wait(&status)` ist äquivalent zu `waitpid(-1, &status, 0);`

- Parameter `int pid`

```
-1    : meaning wait for any child process
>0    : meaning wait for the child whose process ID is equal to the value of pid
```

Weiteres im manual `man 2 waitpid`

```
1 waitpid(15, ...) /* warten auf Kindprozess mit PID 15*/
2 waitpid(-1, ...) /* warten auf irgendein Kindprozess. Returnwert: Kind-PID*/
```

WARTEN: `waitpid()` UND `wait()`

Statusinformationen können via Makros abgefragt werden.

`WIFEXITED(status)` true, falls Kind via `exit()` oder `_exit()` terminiert ist

`WEXITSTATUS(status)` Gibt den Returnwert von `exit()` oder `_exit()` zurück. Nur verwenden falls `WIFEXITED==true!`

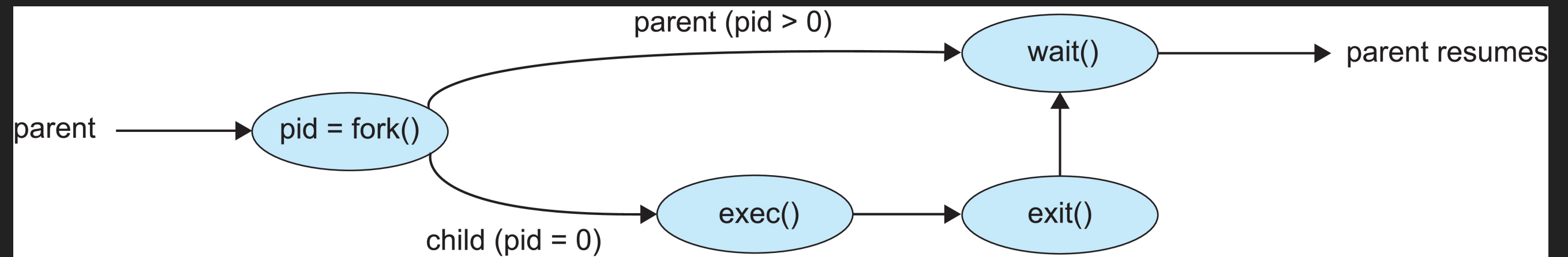
`WIFSIGNALED(status)` true, falls Kind ein Signal erhalten hat

`WTERMSIG(status)` Gibt die Signalnummer des empfangenen Signals zurück. Nur verwenden falls `WIFSIGNALED==true`

Weiteres im Manual `man 2 waitpid`

```
1 int status;
2 waitpid(-1, &status, 0);
3 if(WIFEXITED(status)){
4     printf("child terminated via exit(%d)\n", WEXITSTATUS(status));
5 } else if(WIFSIGNALED(status)) {
6     printf("child terminated via signal [%d]\n", WTERMSIG(status));
7 } else {
8     printf("something different happened to the child!\n");
9 }
```

fork + exec + waitpid



fork + exec + waitpid

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>

void main(void){
    pid_t  pid;
    if ((pid = fork()) == -1) {
        /* error */
        perror("Fork failed: ");
        exit(1);
    } else if (pid == 0) {
        /* child */
        char *arguments[]={"ls", "-l", "-a", "/tmp", NULL};
        printf("child:  PID [%i] PPID [%i]\n", getpid(), getppid());
        execvp(arguments[0], arguments);
        perror("Command failed: ");
        exit(1);
    } else {
        /* parent */
        int status;
        printf("parent: PID [%i] CPID [%i]\n", getpid(), pid);
        waitpid(-1, &status, 0);
        if(WIFEXITED(status)){
            printf("child terminated via exit(%d)\n", WEXITSTATUS(status));
        } else {
            printf("something different happened to the child!\n");
        }
    }
}
```

fork + exec + waitpid VARIANTE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>

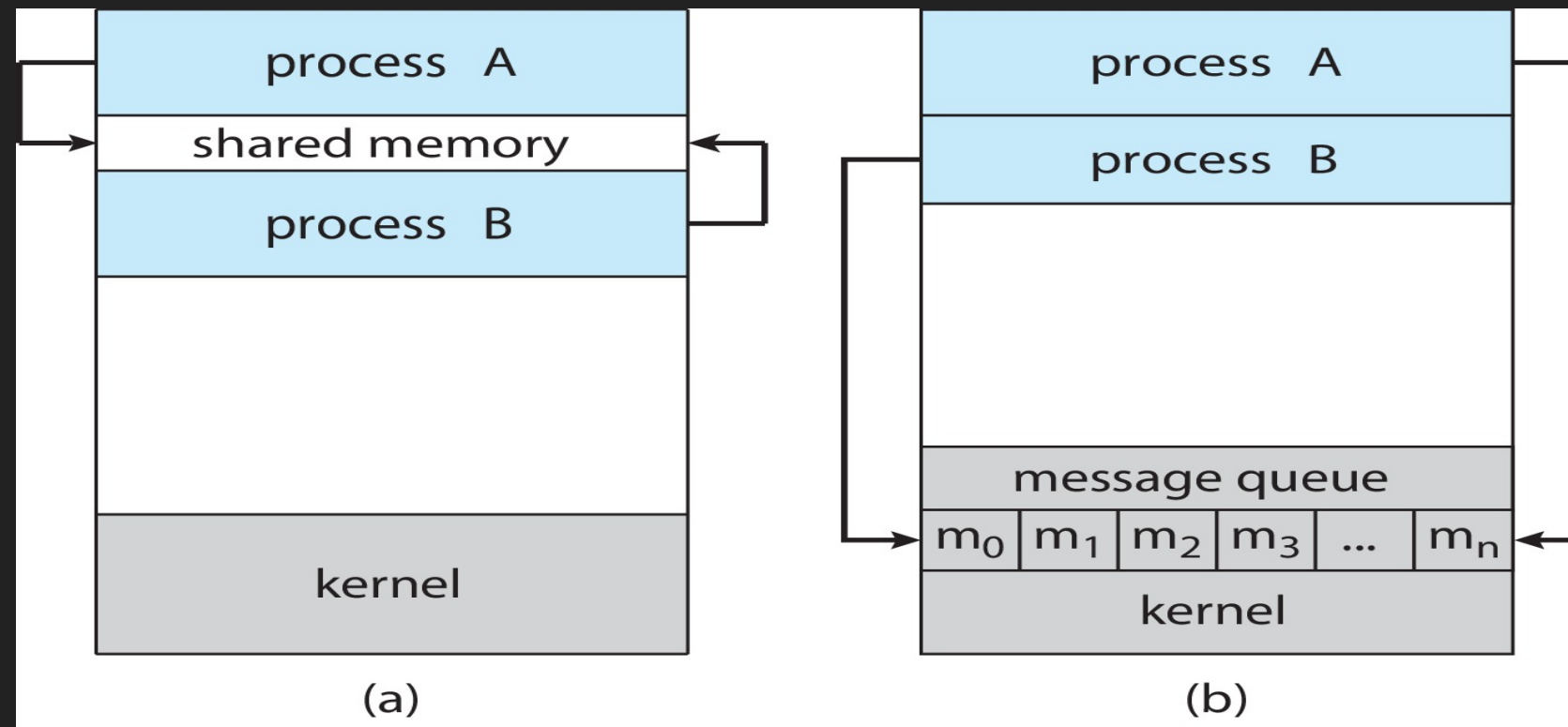
void main(void){
    pid_t pid, wpid;
    if ((pid = fork()) == -1) { /* error */
        perror("Fork failed: ");
        exit(1);
    } else if (pid == 0) { /* first child */
        char *arguments[]={"ls", "-l", "-a", NULL};
        printf("child: PID [%i] PPID [%i]\n", getpid(), getppid());
        execvp(arguments[0], arguments);
        perror("Command failed: ");
        exit(1);
    } else { /* parent */
        // does nothing here!
    }
    if ((pid = fork()) == -1) { /* error */
        perror("Fork failed: ");
        exit(1);
    } else if (pid == 0) { /* second child */
        char *arguments[]={"ls", "-l", "-a", "/foobar", NULL};
        printf("child: PID [%i] PPID [%i]\n", getpid(), getppid());
        execvp(arguments[0], arguments);
        perror("Command failed: ");
        exit(1);
    } else { /* parent */
        int status;
        printf("parent: PID [%i] CPID [%i]\n", getpid(), pid);
        printf("Waiting for child .... \n");
        for (int i=0; i<2; i++) { // wait for 2 processes!
            wpid=waitpid(-1, &status, 0);
            if(WIFEXITED(status)){
                printf("[%i] child (PID: %i) terminated via exit(%d)\n", i, wpid, WEXITSTATUS(status));
            } else if(WIFSIGNALED(status)) {
                printf("[%i] child (PID: %i) terminated via signal [%d]\n", i, wpid, WTERMSIG(status));
            } else {
                printf("[%i] something different happened to the child!\n",i );
            }
        }
    }
}
```


INTER-PROZESS-KOMMUNIKATION (IPC)

- Prozesse in einem System sind entweder **unabhängig** (independent) oder **kooperativ** (cooperating)
- **Unabhängige Prozesse** können durch die Ausführung eines anderen Prozesses nicht beeinflusst werden und keine anderen Prozess beeinflussen
- **Kooperierende Prozesse** können andere Prozesse beeinflussen oder von anderen Prozessen beeinflusst werden
 - Das betrifft insbesondere die gemeinsamen Nutzung von Daten
- Gründe für kooperierende Prozesse
 - Austausch von Informationen/Daten
 - Modularität
 - Beschleunigung von Berechnungen (z.B. auf Multicore Systemen)

IPC MODELLE

- Kooperierende Prozesse benötigen Mechanismen zur Kommunikation (IPC)
 - Modell 1 (a): Geteilter Speicher (shared memory)
 - Modell 2 (b): Nachrichtenübertragung (message passing)



PRODUCER-CONSUMER PROBLEM

Paradigma für kooperierende Prozess — Ein Erzeuger-Prozess erzeugt Daten, die von Verbraucher-Prozess(en) verwendet werden

- **Unbegrenzte Puffer** (unbounded-buffer)
 - Speicher ist (praktisch) nicht begrenzt — Listen, Stacks, Pipes
- **Begrenzte Puffer** (bounded-buffer)
 - Speicher ist explizit begrenzt — Puffer mit fester Größe, Ringpuffer



Auch bei unbegrenzten Puffern existieren natürlich physische Grenzen. Wenn der Speicher voll ist, ist der Speicher voll. **Unbegrenzt** bedeutet also in dem Zusammenhang eine rein logische Annahme. Das Betriebssystem muss hier u.U. intervenieren (z.B. RAM voll → SWAP verwenden).

GETEILTER SPEICHER (SHARED MEMORY)

- Geteilter Speicher ist ein Speicherbereich, der von Erzeuger und Konsument gemeinsam verwendet wird
- Die eigentliche Kommunikation (schreiben/lesen von Daten) muss von den Prozessen explizit gesteuert werden
 - Das Betriebssystem stellt nur den Speicher bereit und sorgt nicht implizit für die eigentliche Synchronisation
- Ein Hauptproblem ist die Bereitstellung von Mechanismen, die eine Synchronisation ermöglichen
 - Diese Mechanismen stellt das Betriebssystem bereit (wird später intensiv besprochen!)

BEISPIEL: BEGRENZTER PUFFER (PRINZIP)

GETEILTE DATEN - PRODUCER

```
#define BUFFER_SIZE 10

typedef struct {
    ...
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

GETEILTE DATEN - CONSUMER

```
#define BUFFER_SIZE 10

typedef struct {
    ...
} item;

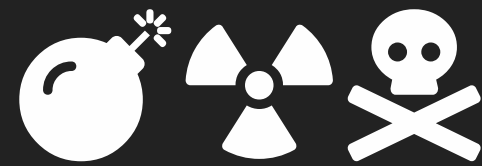
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in next consumed */
}
```



Nehmen Sie an das sich `buffer`, `in` und `out` in einem geteilten Speicher befinden!

BEISPIEL: PIPES IN LINUX (GETEILTER SPEICHER)

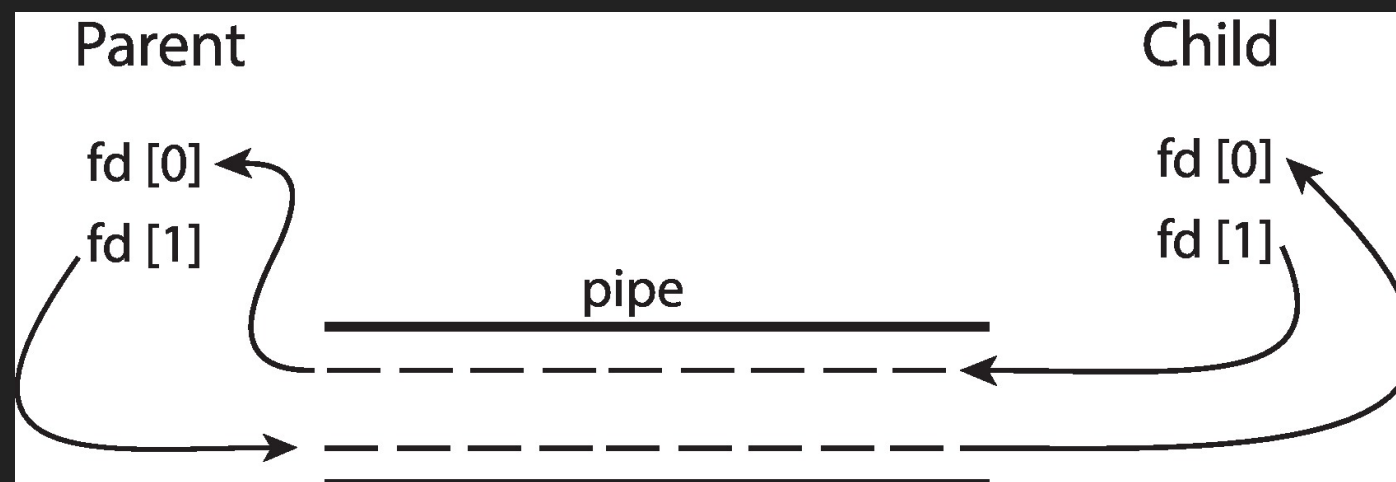


AUFPASSEN!



PIPES

- Normale Pipes ermöglichen das Kommunizieren im üblichen Producer-Consumer-Stil
 - Der Erzeuger schreibt in ein Ende der Pipe
 - Der Empfänger liest vom anderen Ende der Pipe
- Normale Pipes sind **unidirektional** und erfordern eine **Eltern-Kind-Beziehung** bzw. eine **Kind-Kind-Beziehung** (bei gleichem Eltern-Prozess) zwischen den kommunizierenden Prozessen



PIPES

Pipes sind auf der Shell ein sehr mächtiges Werkzeug. Sie erlauben die Aneinanderreihung von beliebig vielen Befehlen, sog. Filtern!



Durch die Verwendung von Pipes (|) lassen sich Aufgaben lösen, die ansonsten sehr spezielle Programme benötigen würden.

Beispiel: Wie oft wurde der Befehl `cd` mit unterschiedlichen Argumenten aufgerufen?

```
$ history | cut -f 7- -d " " | egrep "^cd" | sort | uniq | wc
136      274      2547
```


BENANNTTE PIPES (NAMED PIPES)

- Named Pipes sind bidirektional
 - Beide involvierten Prozesse können sowohl lesend als auch schreibend darauf zugreifen
- Werden als gewöhnliche Datei angelegt
 - `mkfifo <NAME>`
- Benötigen **keine Eltern-Kind-Beziehung**

Beispiel:

```
$ mkfifo /tmp/my_pipe
$ cat < /tmp/my_pipe > /tmp/foo &
[1] 927
$ echo "HELLO CAT PROCESS I AM WRITING TO YOU VIA A NAMED PIPE" > /tmp/my_pipe
[1]+  Done  cat < /tmp/my_pipe > /tmp/foo
$ cat /tmp/foo
HELLO CAT PROCESS I AM WRITING TO YOU VIA A NAMED PIPE
```


VERWENDUNG VON PIPES IN C

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>
#include <string.h>

#define READ_END 0
#define WRITE_END 1

void main(void){
    pid_t pid;
    int pfd[2]; /* Pipe file descriptors */ ❶
    size_t num_read;
    char buf[1024]={0};
    if (pipe2(pfd) == -1) { /* Create the pipe */ ❷
        perror("Pipe failed: "); exit(1);
    }
    if ((pid = fork()) == -1) { /* fork error */
        perror("fork failed"); exit(1);
    } else if (pid == 0) { /* fork child */
        close(pfd[WRITE_END]); ❸
        for (;;) {
            memset(buf, 0, sizeof(buf));
            if((num_read=read(pfd[READ_END], buf, 1024))==-1) ❹
                exit(1);
            else if (num_read == 0)
                break;
            else
                printf("  child read: [%s]\n", buf);
        }
        exit(0);
    } else { /* fork parent */
        close(pfd[READ_END]); ❺
        for (int i=0; i<50;i++){
            sprintf(buf, "%i ", i);
            printf("parent write: [%s]\n", buf);
            if (write(pfd[WRITE_END], buf, strlen(buf)) != strlen(buf)) ❻
                perror("parent write error");
        }
        if (close(pfd[WRITE_END]) == -1) /* Child will see EOF */ ❼
            perror("parent close pipe");
        wait(NULL);
    }
}
```

- ❶ Anlegen der Filedeskriptoren der Pipe
- ❷ Erzeugung der Pipe
- ❸ Kind: Schließen der Ausgabeseite der Pipe (nicht benötigt im Kind)
- ❹ Kind: Einlesen der Pipeeingabe in Buffer
- ❺ Elternprozess: Schließen der Eingabeseite der Pipe (nicht benötigt im Elternprozess)
- ❻ Elternprozess: Schreiben in die Eingabeseite der Pipe
- ❼ Elternprozess: Schließen der Pipe → Erzeugt EOF im Kind und aktiviert Schleifenabbruch

AUSGABE (PIPES IN C)

```
$ gcc -o fork_pipe fork_pipe.c; ./fork_pipe
```

```
parent write: [0 ]
```

```
parent write: [1 ]
```

```
parent write: [2 ]
```

```
parent write: [3 ]
```

```
parent write: [4 ]
```

```
parent write: [5 ]
```

```
parent write: [6 ]
```

```
parent write: [7 ]
```

```
parent write: [8 ]
```

```
parent write: [9 ]
```

```
parent write: [10 ]
```

```
parent write: [11 ]
```

```
parent write: [12 ]
```

```
parent write: [13 ]
```

```
    child read: [0 1 2 3 4 ]
```

```
    child read: [5 6 7 8 9 10 11 12 13 ]
```

```
parent write: [14 ]
```

```
parent write: [15 ]
```

```
    child read: [14 ]
```

```
parent write: [16 ]
```

```
    child read: [15 ]
```

```
parent write: [17 ]
```

```
parent write: [18 ]
```

```
    child read: [16 17 ]
```

```
parent write: [19 ]
```

```
    child read: [18 ]
```

```
    child read: [19 ]
```

VERWENDUNG VON PIPES IN DEN HAUSÜBUNGEN

- In der Hausübung müssen Sie anstatt `pipe(pfd)`, die Funktion `pipe2(pfd, O_CLOEXEC)` verwenden.
 - Dies dient dazu, dass wenn mehrere Pipes offen sind und die Prozesse untereinander kommunizieren, dann werden die Pipes automatisch geschlossen, wenn ein Prozess terminiert.
 - Das hat wiederum zur Folge, dass alle anderen Prozesse ebenfalls terminieren.
- Achtung: In MACOS gibt es die Funktion `pipe2` nicht. Hier muss mit einer Kombination aus `pipe(pfd)` und `fcntl(pfd[0], F_SETFD, FD_CLOEXEC);` und `fcntl(pfd[1], F_SETFD, FD_CLOEXEC);` gearbeitet werden.
 - Der Header `#include <fcntl.h>` ist hierbei zu verwenden.

WEITERE KOMMUNIKATIONSMECHANISMEN

NACHRICHTENÜBERTRAGUNG (MESSAGE PASSING)

- Hier stellt das Betriebssystem Mechanismen zur Verfügung damit Prozesse synchronisiert kommunizieren können
 - Prozesse kommunizieren miteinander ohne direkt auf gemeinsame Variablen zurückzugreifen (kein gemeinsamer Speicher im Userspace!)
- Operationen:
 - `send(message)`
 - `receive(message)`
- Größe der Nachrichten ist entweder begrenzt oder unbegrenzt

NACHRICHTENÜBERTRAGUNG (MESSAGE PASSING)

- Implementierung einer Kommunikationsverbindung (communication link) im Betriebssystem
- Physisch (interessiert uns vorerst nicht):
 - Geteilter Speicher
 - Hardware Bus
 - Netzwerk
- Logisch:
 - **Direkt oder indirekt**
 - **Synchron oder asynchron**
 - Automatisches oder explizites puffern

DIREKTE KOMMUNIKATION

- Bei direkter Kommunikation müssen die Prozesse untereinander bekannt sein:
 - `send(P, message)` — sende Nachricht an Prozess P
 - `receive(Q, message)` — empfangen Nachricht von Prozess Q
- Eigenschaften der Verbindung:
 - Links werden automatisch vom BS erzeugt
 - Ein Link ist genau einem Paar von Kommunikationsprozessen zugeordnet
 - Link kann unidirektional oder bidirektional sein
- **Nachteil:** BS muss u.U. bei direkter Kommunikation sehr viele Links erzeugen und verwalten

INDIREKTE KOMMUNIKATION

- Nachrichten werden mit Hilfe von Postfächern (sog. Mailboxen, auch Ports genannt) gesendet und empfangen
 - Jede Mailbox hat eine eindeutige ID
 - Wenn die Mailbox zwischen den Prozessen geteilt ist, kann Kommunikation erfolgen
- Eigenschaften:
 - Links werden automatisch erzeugt wenn eine Mailbox geteilt wird
 - Eine Mailbox kann mehrere Sender oder Empfänger haben
 - Link kann unidirektional oder bidirektional sein

INDIREKTE KOMMUNIKATION

- Operationen
 - Erzeuge neue Mailbox (port) `create()`
 - Senden `send()` und Empfangen `receive()` mit Hilfe der Mailbox
 - Löschen der Mailbox `destroy()`
- Unter Annahme einer erzeugten Mailbox `M` (`create(M)`)
 - `send(M, message)`
 - `receive(M, message)`

INDIREKTE KOMMUNIKATION

Nehmen Sie an P_1 , P_2 und P_3 teilen sich eine Mailbox M

- P_1 sendet eine Nachricht; P_2 und P_3 empfangen
- Wer erhält nun die Nachricht? P_2 oder P_3 ?

Lösungen:

- Erlaube maximal 2 Prozessen sich eine Mailbox zu teilen
- Erlaube nur einen Prozess zu einer bestimmten Zeit zu empfangen (mutual exclusion)
- System entscheidet (beliebig) wer die Nachricht empfängt. U.U. wird der Sender benachrichtigt wer der Empfänger war

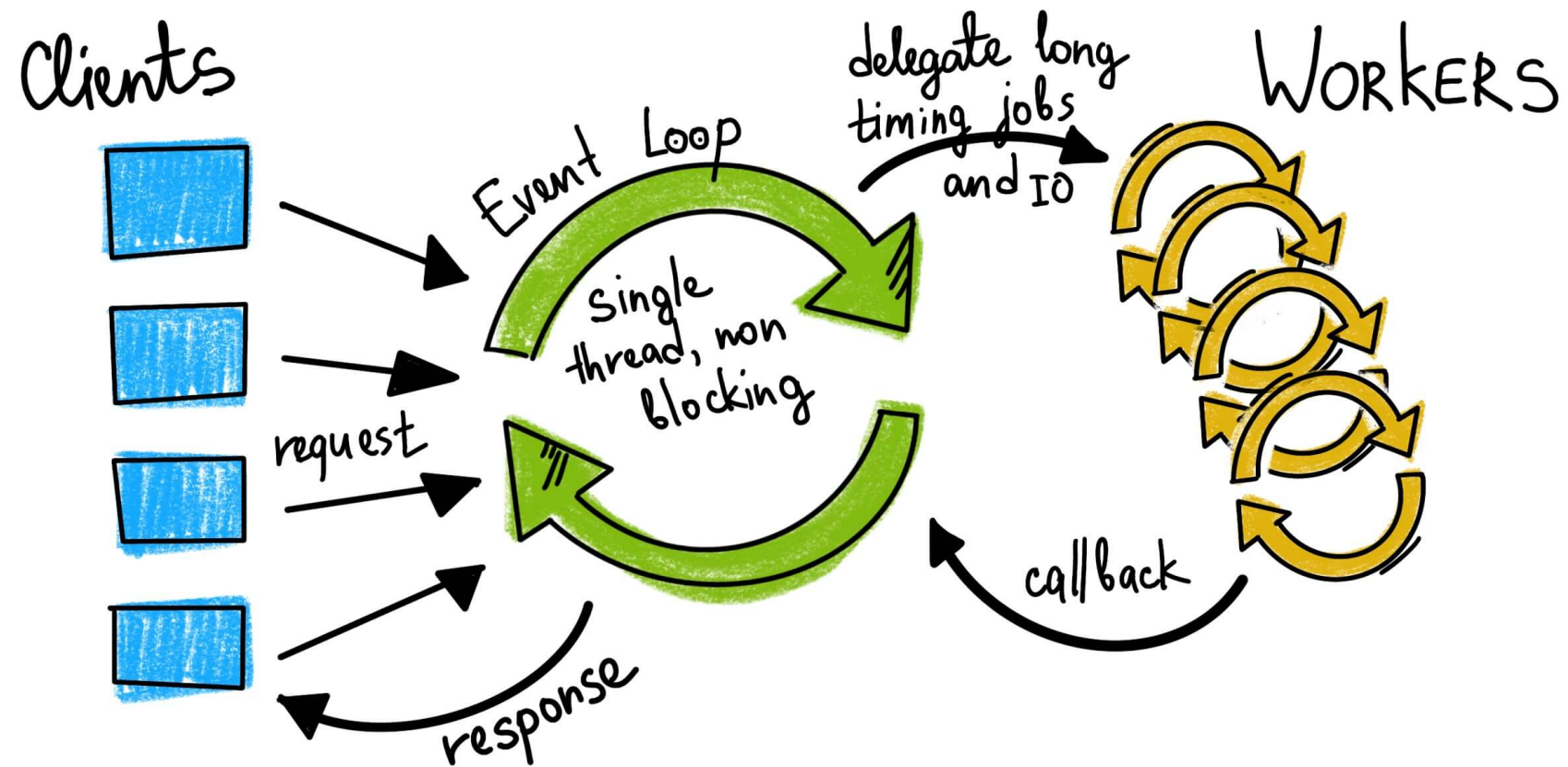
BLOCKIERENDE NACHRICHTENÜBERMITTLUNG

- Blockieren (blocking) → synchron
 - Blockierendes Senden — Der Sender blockiert bis die Nachricht empfangen wurde
 - Blockierendes Empfangen — Der Empfänger blockiert bis eine Nachricht verfügbar ist

SYNCHRONISATION BEI NACHRICHTENÜBERMITTLUNG

- Nicht-Blockierend (non-blocking) → asynchron
 - Nicht-Blockierendes Senden — der Sender sendet eine Nachricht und macht normal weiter
 - Nicht-Blockierendes Empfangen — Empfänger empfängt eine Nachricht oder nichts (Rückkehr mit Fehlermeldung)
 - Erlaubt z.B. die Prüfung auf Nachrichten in (mehreren) Mailboxen in einer Endlosschleife
 - Etwaige Wartelogik muss selbst implementiert werden

NICHT-BLOCKIERENDER EMPFANG



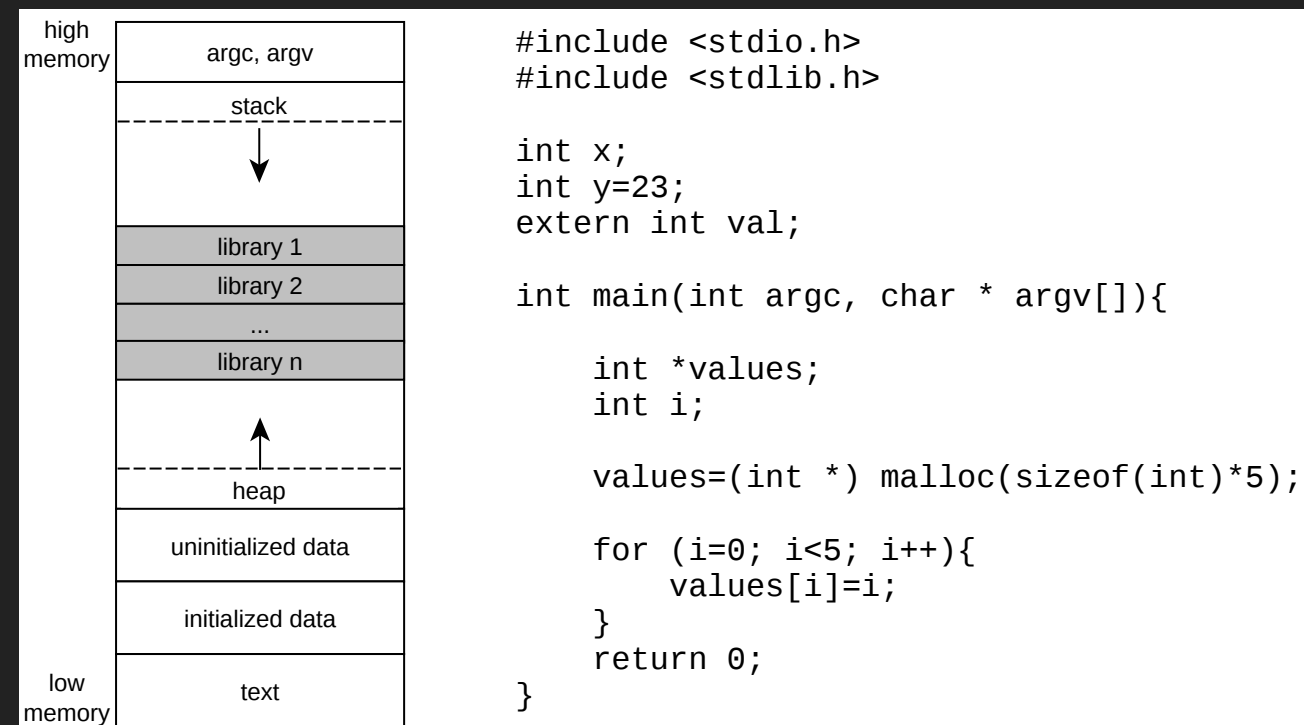
NACHBEARBEITUNG: AUFGABEN UND FRAGEN

PROZESSE 1

Beschreiben Sie in eigenen Worten was der Unterschied zwischen eine Prozess und einem Programm ist

SPEICHERVERWALTUNG

Identifizieren sie die essentiellen in der Grafik angegebenen Programmteile und ordnen Sie diese Programmteile und die Variablen einem Speicherbereich in der Grafik zu.



COMPILER, LINKER, LOADER

Beschreiben Sie kurz und in eigenen Worten welche Schritte von welchem Programm ausgeführt werden müssen um ein Programm, das im Sourcecode vorliegt, schlussendlich auszuführen.

PROZESSZUSTÄNDE

Zeichnen Sie ein Diagramm, das alle Prozesszustände und die damit verbundenen Aktionen zeigt (Inklusive `new` und `terminated`). Geben Sie für wartende I/O Events hierzu mindestens 3 Beispiele an, worauf potentiell gewartet werden könnte.

KONTEXTWECHSEL

Beschreiben Sie kurz welche Aufgaben der Betriebssystemkernel von Linux bei einem Kontextwechsel ausführen muss. Gehen Sie dabei auf folgende Punkt ein.

- Wo werden die Daten bei einem Kontextwechsel gespeichert
- Welche Daten müssen unbedingt vom Betriebssystemkernel gespeichert werden
- In welchen Fällen werden Kontextwechsel ausgeführt und warum ist dies überhaupt nötig?
 - Geben Sie mindestens 2 Beispiele an, die einen Kontextwechsel erfordern.
- Was hat der Prozesskontrollblock für eine Aufgabe und wie wird dieser bei einem Kontextwechsel verwendet?

PROZESSERZEUGUNG

Schreiben Sie ein kurzes Programm das eine `fork/exec` Semantik verwendet um das Programm `ls` mit den Parametern `-l` auzurufen.

PROZESSERZEUGUNG 2

Geben Sie an was das folgende Programm ausgibt:

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <wait.h>
int value = 5;
int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) { /* child process */
        value += 15;
        printf("CHILD: value = %d\n",value);
        return 0;
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d\n",value);
        return 0;
    }
}
```

PROZESSERZEUGUNG 2

- Beschreiben Sie kurz und in eigenen Worten, was die Systemaufrufe `fork()`, `exec*` und `waitpid()` tun.
- Nennen Sie mindestens 2 Varianten von `exec()` und benennen Sie die Unterschiede.
- Warum haben wir in den Übungen nicht anstelle von `exec()` den Systemaufruf `system()` verwendet?

KOMMUNIKATION

Geben Sie 2 Beispiele an, wie mehrere Prozess miteinander kommunizieren können.

KOMMUNIKATION 2

- Erläutern Sie kurz was eine sog. `pipe` ist und wobei Sie verwendet werden kann. Geben Sie weiterhin mindestens 3 sinnvolle Beispiele an, die bei der Verwendung von einer `pipe` auf einer Shell sinnvoll wären.
- Welche Voraussetzungen müssen erfüllt sein, damit 2 oder mehr Prozesse mittels einer `Pipe` kommunizieren können?
- Wann macht der Einsatz einer unbenannten `Pipe` Sinn und wann sollte man eine benannte `Pipe` verwenden?